FastQC (Raw data)

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=8
#SBATCH --job-name="FastQC"
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

# ---------------------------------
# Fail safely
# ---------------------------------
set -e
set -o pipefail

# ---------------------------------
# Add FastQC to PATH
# ---------------------------------
export PATH="/opt/exp_soft/FastQC-0.12.1:$PATH"

# ---------------------------------
# Defining directories
# ---------------------------------
base_input_dir="/scratch/[redacted]/raw_data"
output_base_dir="/scratch/[redacted]/RNA_seq_analysis/fast_qc/raw"

mkdir -p "$output_base_dir"

echo "Starting FastQC pipeline on compressed FASTQ files..."

# ------------------------------------------------
# Loop through each subdirectory inside raw_data
# ------------------------------------------------
for sample_dir in "$base_input_dir"/*/; do

    # Skip if not a directory
    [ -d "$sample_dir" ] || continue

    # Extract sample/folder name
    sample_name=$(basename "$sample_dir")
    echo "Processing folder: $sample_name"

    # Create output folder for this sample
    sample_output_dir="$output_base_dir/$sample_name"
    mkdir -p "$sample_output_dir"

    # ---------------------------------
    # Run FastQC directly on .fq.gz files
    # ---------------------------------
    for fq_file in "$sample_dir"/*.fq.gz; do
        [ -f "$fq_file" ] || continue
```

```
        fq_basename=$(basename "$fq_file")
        html_out="$sample_output_dir/${sample_name}_${fq_basename}_fastqc.html"
        zip_out="$sample_output_dir/${sample_name}_${fq_basename}_fastqc.zip"

        # Skip if outputs already exist
        if [ -f "$html_out" ] && [ -f "$zip_out" ]; then
            echo "Skipping FastQC, output already exists for $fq_basename"
            continue
        fi

        # Run FastQC
        echo "Running FastQC on $fq_file"
        fastqc -t 8 -o "$sample_output_dir" "$fq_file"

        # Rename FastQC outputs to include folder/sample name
        # Find generated files in case FastQC changed names slightly
        gen_html=$(find "$sample_output_dir" -maxdepth 1 -type f -name "*_fastqc.html" | grep
-F "$fq_basename" || true)
        gen_zip=$(find "$sample_output_dir" -maxdepth 1 -type f -name "*_fastqc.zip" | grep -F
"$fq_basename" || true)

        if [ -f "$gen_html" ]; then
            mv "$gen_html" "$html_out"
        fi
        if [ -f "$gen_zip" ]; then
            mv "$gen_zip" "$zip_out"
        fi

    done

done

echo "FASTQC analysis completed successfully!"
```

Adapter sequences

>1
AGACTAAGGAGACAGATTGG
>2
GCGTGATCGTGAATTGGCCG
>3
CACACTGATGACGCATCAAT
>4
TTAGAGCTACCGTGACAGTC
>5
AATCACTTCATTAACTTGAA
>6
CCGATCACAGACGCGACCGT
>7
TGTGCGCGTCTGTGTCATCA
>8
GTCTGTGACTCTCTCGGATC
>9
CCGCTCAGTA
>10
GGTGTGTACA
>11
TTCACGTAAG
>12
GGTTCCACAC
>13
AGGTATTCTT
>14
CGAATGCAAC
>15
TTCAACGGCG
>16
CTCGGCGGAA
>17
ACGGTAATGG
>18
GATCCGACGT
>19
TCACGATACA
>20
GATTCTCTTC
>21
AGAATTAATG
>22
ACCAGCGTCA
>23
CGTCAGGCTC
>24
TAGTACAGCAAGCTCTGTGT
>25
GGATGTGTACCTGACGTTAC
>26
ACCATGTATGGAACGCAGTG
>27

CTTCCAGCGTTCTGAACCGA
>28
AACGCGCTGGAAGCTGAACC
>29
CCGAACTCATCCTATATACA
>30
TGTCGTAGTCTTCGGTCGTG
>31
GTAGTACACAGGATACGCAT
>32
GACAGATTGGAGACTAAGGA
>33
GAATTGGCCGGCGTGATCGT
>34
ACGCATCAATCACACTGATG
>35
CGTGACAGTCTTAGAGCTAC
>36
TTAACTTGAAAATCACTTCA
>37
ACGCGACCGTCCGATCACAG
>38
TGTGTCATCATGTGCGCGTC
>39
CTCTCGGATCGTCTGTGACT
>40
AGCTCTGTGTTAGTACAGCA
>41
CTGACGTTACGGATGTGTAC
>42
GAACGCAGTGACCATGTATG
>43
TCTGAACCGACTTCCAGCGT
>44
AAGCTGAACCAACGCGCTGG
>45
CCTATATACACCGAACTCAT
>46
TTCGGTCGTGTGTCGTAGTC
>47
GGATACGCATGTAGTACACA

Trimmomatic

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=8
#SBATCH --job-name="FastQC"
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

# ---------------------------------
# Fail safely
# ---------------------------------
set -e
set -o pipefail

# ---------------------------------
# Add FastQC to PATH
# ---------------------------------
export PATH="/opt/exp_soft/FastQC-0.12.1:$PATH"

# ---------------------------------
# Directories
# ---------------------------------
base_input_dir="/scratch/[redacted]/RNA_seq_analysis/trimmed"
output_base_dir="/scratch/[redacted]/RNA_seq_analysis/fast_qc/trimmed"

mkdir -p "$output_base_dir"

echo "Starting FastQC pipeline on compressed FASTQ files..."

# ------------------------------------------------
# Loop through each subdirectory inside raw_data
# ------------------------------------------------
for sample_dir in "$base_input_dir"/*/; do

    # Skip if not a directory
    [ -d "$sample_dir" ] || continue

    # Extract sample/folder name
    sample_name=$(basename "$sample_dir")
    echo "Processing folder: $sample_name"

    # Create output folder for this sample
    sample_output_dir="$output_base_dir/$sample_name"
    mkdir -p "$sample_output_dir"

    # ---------------------------------
    # Run FastQC directly on .fq.gz files
    # ---------------------------------
    for fq_file in "$sample_dir"/*.fq.gz; do
```

```
        [ -f "$fq_file" ] || continue

        fq_basename=$(basename "$fq_file")
        html_out="$sample_output_dir/${sample_name}_${fq_basename}_fastqc.html"
        zip_out="$sample_output_dir/${sample_name}_${fq_basename}_fastqc.zip"

        # Skip if outputs already exist
        if [ -f "$html_out" ] && [ -f "$zip_out" ]; then
            echo "Skipping FastQC, output already exists for $fq_basename"
            continue
        fi

        # Run FastQC
        echo "Running FastQC on $fq_file"
        fastqc -t 8 -o "$sample_output_dir" "$fq_file"

        # Rename FastQC outputs to include folder/sample name
        # Find generated files in case FastQC changed names slightly
        gen_html=$(find "$sample_output_dir" -maxdepth 1 -type f -name "*_fastqc.html" | grep
-F "$fq_basename" || true)
        gen_zip=$(find "$sample_output_dir" -maxdepth 1 -type f -name "*_fastqc.zip" | grep -F
"$fq_basename" || true)

        if [ -f "$gen_html" ]; then
            mv "$gen_html" "$html_out"
        fi
        if [ -f "$gen_zip" ]; then
            mv "$gen_zip" "$zip_out"
        fi

    done

done

echo "FASTQC analysis completed successfully!"
```

FastQC (Trimmed files)

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=8
#SBATCH --job-name="FastQC"
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

# ----------------------------------
# Fail safely
# ----------------------------------
set -e
set -o pipefail

# ----------------------------------
# Add FastQC to PATH
# ----------------------------------
export PATH="/opt/exp_soft/FastQC-0.12.1:$PATH"

# ----------------------------------
# Directories
# ----------------------------------
base_input_dir="/scratch/[redacted]/RNA_seq_analysis/trimmed"
output_base_dir="/scratch/[redacted]/RNA_seq_analysis/fast_qc/trimmed"

mkdir -p "$output_base_dir"

echo "Starting FastQC pipeline on compressed FASTQ files..."

# ------------------------------------------------
# Loop through each subdirectory inside raw_data
# ------------------------------------------------
for sample_dir in "$base_input_dir"/*/; do

    # Skip if not a directory
    [ -d "$sample_dir" ] || continue

    # Extract sample/folder name
    sample_name=$(basename "$sample_dir")
    echo "Processing folder: $sample_name"

    # Create output folder for this sample
    sample_output_dir="$output_base_dir/$sample_name"
    mkdir -p "$sample_output_dir"

    # ----------------------------------
    # Run FastQC directly on .fq.gz files
    # ----------------------------------
    for fq_file in "$sample_dir"/*.fq.gz; do
        [ -f "$fq_file" ] || continue
```

```
        fq_basename=$(basename "$fq_file")
        html_out="$sample_output_dir/${sample_name}_${fq_basename}_fastqc.html"
        zip_out="$sample_output_dir/${sample_name}_${fq_basename}_fastqc.zip"

        # Skip if outputs already exist
        if [ -f "$html_out" ] && [ -f "$zip_out" ]; then
            echo "Skipping FastQC, output already exists for $fq_basename"
            continue
        fi

        # Run FastQC
        echo "Running FastQC on $fq_file"
        fastqc -t 8 -o "$sample_output_dir" "$fq_file"

        # Rename FastQC outputs to include folder/sample name
        # Find generated files in case FastQC changed names slightly
        gen_html=$(find "$sample_output_dir" -maxdepth 1 -type f -name "*_fastqc.html" | grep
-F "$fq_basename" || true)
        gen_zip=$(find "$sample_output_dir" -maxdepth 1 -type f -name "*_fastqc.zip" | grep -F
"$fq_basename" || true)

        if [ -f "$gen_html" ]; then
            mv "$gen_html" "$html_out"
        fi
        if [ -f "$gen_zip" ]; then
            mv "$gen_zip" "$zip_out"
        fi

    done

done

echo "FASTQC analysis completed successfully!"
```

rRNA contamination

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=8
#SBATCH --job-name="rRNA_counting"
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

set -euo pipefail

# ----------------------------
# Load tools
# ----------------------------
export PATH="/opt/exp_soft/bedtools-2.31.0:$PATH"
export PATH="/opt/exp_soft/bowtie2-2.5.4:$PATH"

# ------------------------------------------------
# Checking tool versions
# ------------------------------------------------
echo "Checking tool versions..."
bedtools --version
bowtie2 --version

# ----------------------------
# Input files and threads
# ----------------------------
GTF="/scratch/[redacted]/RNA_seq_analysis/ref_genome/hg38_rRNA_only.gtf"
GENOME="/scratch/[redacted]/RNA_seq_analysis/ref_genome/hg38.fa"
THREADS=8

# ----------------------------
# Base output directory
# ----------------------------
OUTBASE="/scratch/[redacted]/RNA_seq_analysis/rRNA_qc"
mkdir -p "$OUTBASE"

# ----------------------------
# Sample list
# ----------------------------
SAMPLES=("[redacted]" "[redacted]" "[redacted]" "[redacted]" "[redacted]" "[redacted]"
"[redacted]" "[redacted]" "[redacted]" "[redacted]" "[redacted]" "[redacted]" "[redacted]")

# ----------------------------
# Loop over samples
# ----------------------------
for SAMPLE in "${SAMPLES[@]}"; do

    echo "====================================="
    echo "Processing sample: $SAMPLE"
    echo "====================================="
```

```
    # ----------------------------
    # Input files
    # ----------------------------

R1="/scratch/[redacted]/RNA_seq_analysis/trimmed/${SAMPLE}/${SAMPLE}_R1_paired.fq.
gz"

R2="/scratch/[redacted]/RNA_seq_analysis/trimmed/${SAMPLE}/${SAMPLE}_R2_paired.fq.
gz"

    # ----------------------------
    # Output directory
    # ----------------------------
    OUTDIR="${OUTBASE}/${SAMPLE}"
    mkdir -p "$OUTDIR"

    BED="${OUTDIR}/rRNA.bed"
    BED_MERGED="${OUTDIR}/rRNA_merged.bed"
    FASTA="${OUTDIR}/rRNA.fa"
    INDEX="${OUTDIR}/rRNA_index"
    LOG="${OUTDIR}/${SAMPLE}_rRNA_alignment.log"
    SUMMARY="${OUTDIR}/${SAMPLE}_rRNA_summary.txt"

    # ----------------------------
    # Check input files
    # ----------------------------
    if [[ ! -f "$R1" || ! -f "$R2" ]]; then
        echo "ERROR: Missing FASTQ files for $SAMPLE"
        exit 1
    fi

    # ----------------------------
    # 1. Extract rRNA regions
    # ----------------------------
    echo "[$SAMPLE] Extracting rRNA regions..."

    awk 'BEGIN{FS="\t"; OFS="\t"}
    $0 ~ /gene_name \"RRNAD/ {
        print $1, $4-1, $5, ".", 0, $7
    }' "$GTF" > "$BED"

    if [[ ! -s "$BED" ]]; then
        echo "ERROR: BED file empty for $SAMPLE"
        exit 1
    fi

    # ----------------------------
    # 2: Merge regions
    # ----------------------------
    bedtools sort -i "$BED" | bedtools merge -s -i - > "$BED_MERGED"

    # ----------------------------
    # 3: Create FASTA
    # ----------------------------
```

```
    bedtools getfasta \
        -fi "$GENOME" \
        -bed "$BED_MERGED" \
        -fo "$FASTA" \
        -s

    # Safety check (prevents Bowtie2 crash)
    if [[ ! -s "$FASTA" ]]; then
        echo "ERROR: FASTA is empty for $SAMPLE"
        exit 1
    fi

    # ----------------------------
    # 4: Build index
    # ----------------------------
    bowtie2-build "$FASTA" "$INDEX"

    # ------------------------------------------------
    # 5: Align reads (no alignment file needed)
    # ------------------------------------------------
    echo "[$SAMPLE] Aligning reads to rRNA reference..."

    bowtie2 \
        -x "$INDEX" \
        -1 "$R1" \
        -2 "$R2" \
        --very-sensitive \
        -p "$THREADS" \
        --no-unal \
        2> "$LOG"

    # ----------------------------
    # 6: Calculate rRNA %
    # ----------------------------
    exact=$(grep "aligned concordantly exactly 1 time" "$LOG" | awk '{print $1}' | tr -d '()%')
    multi=$(grep "aligned concordantly >1 times" "$LOG" | awk '{print $1}' | tr -d '()%')

    exact=${exact:-0}
    multi=${multi:-0}

    rrna_percent=$(awk "BEGIN {print $exact + $multi}")

    echo "rRNA contamination (%) = $rrna_percent" > "$SUMMARY"

    echo "[$SAMPLE] COMPLETE"

done

echo "======================================"
echo "ALL SAMPLES FINISHED"
echo "OUTPUT: $OUTBASE"
echo "======================================"
```

RNA STAR

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --job-name="STAR_mapping"
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

# ----------------------------
# Setup and configuration
# ----------------------------

# Fail-safe to stop on errors
set -e
set -o pipefail

# Add STAR to PATH
export PATH="/opt/exp_soft/STAR-2.7.11b:$PATH"

# Load samtools
module load software/samtools-1.20

# Threads for STAR
threads=8

# Genome reference
genome_dir="/scratch/[redacted]/RNA_seq_analysis/ref_genome"
fasta_file="${genome_dir}/hg38.fa"
gtf_file="${genome_dir}/hg38.refGene.gtf"
read_length=100
overhang=$((read_length - 1))

# Reads and output directories
reads_dir="/scratch/vhrgra004/RNA_seq_analysis/trimmed/paired"
output_base_dir="/scratch/[redacted]/RNA_seq_analysis/STAR_mapped"
mkdir -p "$output_base_dir"

# ----------------------------
# Check and unzip reference files
# ----------------------------

# Handle FASTA
if [[ "$fasta_file" == *.gz ]]; then
    unzipped_fasta="${fasta_file%.gz}"
    if [[ ! -f "$unzipped_fasta" ]]; then
        echo "Unzipping FASTA: $fasta_file"
        gunzip -c "$fasta_file" > "$unzipped_fasta"
    else
        echo "FASTA already unzipped: $unzipped_fasta"
```

```
    fi
    fasta_file="$unzipped_fasta"
fi

# Handle GTF
if [[ "$gtf_file" == *.gz ]]; then
    unzipped_gtf="${gtf_file%.gz}"
    if [[ ! -f "$unzipped_gtf" ]]; then
        echo "Unzipping GTF: $gtf_file"
        gunzip -c "$gtf_file" > "$unzipped_gtf"
    else
        echo "GTF already unzipped: $unzipped_gtf"
    fi
    gtf_file="$unzipped_gtf"
fi


# -----------------------------
# Generate STAR genome index if needed
# -----------------------------
if [ ! -f "${genome_dir}/SA" ]; then
    echo "Generating STAR genome index..."
    STAR --runThreadN $threads \
         --runMode genomeGenerate \
         --genomeDir "$genome_dir" \
         --genomeFastaFiles "$fasta_file" \
         --sjdbGTFfile "$gtf_file" \
         --sjdbOverhang "$overhang"
else
    echo "Genome index already exists, skipping generation."
fi

# ---------------------
# Align reads
# ---------------------
echo "Starting alignment loop through all samples..."

# Hardcoded sample names
samples=(
    [redacted]
)

# Loop through each sample
```

```
for sample_prefix in "${samples[@]}"; do

    echo "Processing sample: $sample_prefix"

    # Detect trimmed paired files (.fq.gz)
    R1="${reads_dir}/${sample_prefix}_R1_paired.fq.gz"
    R2="${reads_dir}/${sample_prefix}_R2_paired.fq.gz"

    if [[ ! -f "$R1" || ! -f "$R2" ]]; then
        echo "Missing paired reads for $sample_prefix, skipping..."
        continue
    fi

    echo "R1: $R1"
    echo "R2: $R2"

    # Per-sample output folder
    sample_outdir="${output_base_dir}/${sample_prefix}/"
    mkdir -p "$sample_outdir"

    # Run STAR
    STAR --runThreadN $threads \
         --runMode alignReads \
         --genomeDir "$genome_dir" \
         --readFilesIn "$R1" "$R2" \
         --readFilesCommand zcat \
         --outFilterMultimapNmax 10 \
         --outSAMtype BAM SortedByCoordinate \
         --alignEndsType Local \
         --chimSegmentMin 15 \
         --chimJunctionOverhangMin 10 \
         --outSAMattrRGline ID:$sample_prefix \
         --chimOutType SeparateSAMold \
         --twopassMode Basic \
         --outFilterScoreMinOverLread 0.3 \
         --outFilterMatchNminOverLread 0.3 \
         --outFilterMatchNmin 20 \
         --outFilterMismatchNoverReadLmax 0.04 \
         --outReadsUnmapped Fastx \
         --outFileNamePrefix "${sample_outdir}/${sample_prefix}_"

    # Index BAM
    bam_file="${sample_outdir}/${sample_prefix}_Aligned.sortedByCoord.out.bam"
    samtools index "$bam_file"

    # Alignment stats
    samtools flagstat "$bam_file" > "${sample_outdir}/${sample_prefix}_flagstat.txt"

    echo "STAR alignment and BAM indexing completed for $sample_prefix"

    # ----------------------------
    # Process chimeric alignments (if present)
    # ----------------------------
    chimeric_sam="${sample_outdir}/${sample_prefix}_Chimeric.out.sam"
    if [[ -f "$chimeric_sam" ]]; then
```

```
chimeric_bam_unsorted="${sample_outdir}/${sample_prefix}_chimeric_reads.unsorted.bam"
        chimeric_bam="${sample_outdir}/${sample_prefix}_chimeric_reads.bam"

        samtools view -Sb "$chimeric_sam" > "$chimeric_bam_unsorted"
        samtools sort -o "$chimeric_bam" "$chimeric_bam_unsorted"
        rm "$chimeric_bam_unsorted"

        samtools index "$chimeric_bam"
        samtools flagstat "$chimeric_bam" >
"${sample_outdir}/${sample_prefix}_chimeric_stats.txt"

        echo "Processed chimeric alignments for $sample_prefix"
    else
        echo "No chimeric SAM file found for $sample_prefix"
    fi

done

echo "All samples processed successfully!"

# ----------------------------------------
# Moving all BAM files to separate folder
# ----------------------------------------

# Making bam_files directory
mkdir -p /scratch/vhrgra004/RNA_seq_analysis/STAR_mapped/bam_files

# Move all BAM files into bam_files directory
find /scratch/vhrgra004/RNA_seq_analysis/STAR_mapped \
-type f \
-name "*_Aligned.sortedByCoord.out.bam" \
-exec cp {} /scratch/vhrgra004/RNA_seq_analysis/STAR_mapped/bam_files/ \;

echo "All BAM files moved into bam_files directory successfully!"
```

featureCounts

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --job-name="featureCounts"
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

set -e
set -o pipefail

echo "Job started on $(hostname)"

# ----------------------------
# Load software
# ----------------------------
export PATH="/opt/exp_soft/subread-2.0.6/bin:$PATH"

# ----------------------------
# Directories
# ----------------------------
bam_dir="/scratch/[redacted]/RNA_seq_analysis/STAR_mapped/bam_files"
output_dir="/scratch/[redacted]/RNA_seq_analysis/featureCounts"
summaries="$output_dir/alignment_summaries"

gtf_file="/scratch/[redacted]/RNA_seq_analysis/ref_genome/hg38.refGene.gtf"
fasta_file="/scratch/[redacted]/RNA_seq_analysis/ref_genome/hg38.fa"

mkdir -p "$output_dir"
mkdir -p "$summaries"

# ----------------------------
# Collect BAM files
# ----------------------------
echo "Collecting BAM files..."

bam_list=(
"$bam_dir/[redacted]Aligned.sortedByCoord.out.bam"
"$bam_dir/[redacted]Aligned.sortedByCoord.out.bam"
"$bam_dir/[redacted]Aligned.sortedByCoord.out.bam"
"$bam_dir/[redacted]Aligned.sortedByCoord.out.bam"
"$bam_dir/[redacted]Aligned.sortedByCoord.out.bam"
"$bam_dir/[redacted]Aligned.sortedByCoord.out.bam"
"$bam_dir/[redacted]Aligned.sortedByCoord.out.bam"
"$bam_dir/[redacted]Aligned.sortedByCoord.out.bam"
"$bam_dir/[redacted]_Aligned.sortedByCoord.out.bam"
"$bam_dir/[redacted]Aligned.sortedByCoord.out.bam"
"$bam_dir/[redacted]Aligned.sortedByCoord.out.bam"
```

```
"$bam_dir/[redacted]Aligned.sortedByCoord.out.bam"
"$bam_dir/[redacted]Aligned.sortedByCoord.out.bam"
)

echo "Total BAM files: ${#bam_list[@]}"
printf '%s\n' "${bam_list[@]}"

# ------------------------------
# Run featureCounts
# ------------------------------
echo "Running featureCounts..."

featureCounts \
    -a "$gtf_file" \
    -o "$output_dir/combined_featureCounts.txt" \
    -p \
    -T 4 \
    -t exon \
    -g gene_id \
    --countReadPairs \
    "${bam_list[@]}"

echo "featureCounts finished."

# ------------------------------
# Clean matrix
# ------------------------------
input_file="$output_dir/combined_featureCounts.txt"
clean_file="$output_dir/combined_counts_matrix.txt"

echo "Cleaning featureCounts matrix..."

awk 'BEGIN{FS=OFS="\t"}
!/^#/ {
    if(NR==2){
        printf "GeneID"
        for(i=7;i<=NF;i++){
            split($i,a,"/")
            sample=a[length(a)]

            # Remove BAM suffix
            sub("_Aligned.sortedByCoord.out.bam","",sample)

            # Case 1: remove prefix like "17-"
            sub(/^[0-9]+-/, "", sample)

            # Case 2: remove suffix like "_19"
            sub(/_[0-9]+$/, "", sample)

            printf OFS sample
        }
        printf "\n"
    } else {
        printf $1
        for(i=7;i<=NF;i++) printf OFS $i
```

```
        printf "\n"
    }
}' "$input_file" > "$clean_file"

echo "Clean count matrix created:"
echo "$clean_file"

# ----------------------------
# Move summary file
# ----------------------------
mv "$output_dir/combined_featureCounts.txt.summary" "$summaries/"

echo "Pipeline finished successfully."
```

Virtual environment setup

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=03:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --job-name=create_conda_env
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

set -e
set -o pipefail

# ------------------------------
# Load miniconda
# ------------------------------
export PATH="/opt/exp_soft/miniconda3-py3.12/bin:$PATH"
source /opt/exp_soft/miniconda3-py3.12/etc/profile.d/conda.sh
eval "$(conda shell.bash hook)"

# ------------------------------
# Create combined R and Python environment
# ------------------------------
echo "Creating combined Conda environment..."

conda create --prefix /scratch/[redacted]/conda_envs/conda_env \
  -c conda-forge -c bioconda \
  python=3.12 \
  pandas \
  r-base=4.3 \
  r-essentials \
  r-data.table \
  r-ggplot2 \
  r-pheatmap \
  r-dplyr \
  r-readr \
  r-stringr \
  r-purrr \
  r-tibble \
  r-vroom \
  r-matrixstats \
  r-ggrepel \
  r-ggpubr \
  r-tidyverse \
  r-rstatix \
  r-ggsignif \
  bioconductor-deseq2 \
  bioconductor-edger \
  bioconductor-fgsea \
  bioconductor-clusterprofiler \
```

```
  bioconductor-reactomepa \
  bioconductor-org.hs.eg.db \
  bioconductor-enrichplot \
  bioconductor-apeglm \
  bioconductor-enhancedvolcano \
  -y

echo "Environment created successfully!"

# ----------------------------
# Activate environment
# ----------------------------
echo "Activating combined Conda environment..."

conda activate /scratch/[redacted]/conda_envs/conda_env

# ----------------------------
# Install additional CRAN packages
# ----------------------------
echo "Installing additional CRAN packages..."

Rscript -e 'install.packages(c("msigdbr"), repos="https://cloud.r-project.org")'

Rscript -e 'install.packages(c("pheatmap", "matrixStats"), repos="https://cloud.r-project.org")'

# ----------------------------
# Verify R and Python
# ----------------------------
echo "Checking R and Python installations..."

which R
which Rscript
which python

echo "R version:"
R --version

echo "Python version:"
python --version

# ----------------------------
# Verify installed R package versions
# ----------------------------
echo "Checking installed package versions..."

Rscript -e '
cat("\n============================\n")
cat("ENVIRONMENT CHECK\n")
cat("============================\n\n")

cat("R version:\n")
cat(R.version.string, "\n\n")

packages <- c(
  "DESeq2","edgeR","clusterProfiler","ReactomePA",
```

```
  "org.Hs.eg.db","enrichplot","fgsea",
  "data.table","pheatmap","ggplot2",
  "dplyr","readr","stringr","purrr",
  "ggrepel","vroom","tibble","matrixStats","msigdbr",
  "ggpubr","apeglm","EnhancedVolcano",
  "tidyverse","rstatix","ggsignif"
)

for(pkg in packages) {
  if(requireNamespace(pkg, quietly = TRUE)) {
    cat(pkg, ":", as.character(packageVersion(pkg)), "\n")
  } else {
    cat(pkg, ": NOT INSTALLED\n")
  }
}

cat("\n============================\n\n")
'


# ----------------------------
# Verify immune analysis packages
# ----------------------------
echo "Checking immune analysis packages..."

Rscript -e '
cat("\n====================\n")
cat("IMMUNE PACKAGE CHECK\n")
cat("====================\n\n")

pkgs <- c("ggpubr", "rstatix", "ggsignif", "tidyverse")

for (p in pkgs) {
  cat(p, ":", requireNamespace(p, quietly = TRUE), "\n")
}

cat("\n.libPaths():\n")
print(.libPaths())
'


# ----------------------------
# List installed R packages
# ----------------------------
echo "Installed R packages:"
Rscript -e "installed.packages()[,c('Package','Version')]"

echo "Combined Conda environment setup COMPLETE"
```

Pre-DEG quality control PCA plot

BASH script:

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --job-name="preDEG_PCA"
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

set -e
set -o pipefail

echo "Job started on $(hostname)"

# Load miniconda (to get conda command)
export PATH="/opt/exp_soft/miniconda3-py3.12/bin:$PATH"
source /opt/exp_soft/miniconda3-py3.12/etc/profile.d/conda.sh

echo "Activating R conda environment..."
conda activate /scratch/[redacted]/conda_envs/conda_env

# Sanity check
echo "Which R am I using?"
which R
R --version

# Run the R script
echo "Running preDEG_PCA.R..."
Rscript preDEG_PCA.R

echo "Job finished"
```

R script:

```
#!/usr/bin/env Rscript


# ------------------------------------------------------------
# 1. Load required packages
# ------------------------------------------------------------

suppressPackageStartupMessages({
  library(DESeq2)
  library(edgeR)
  library(matrixStats)
  library(ggplot2)
  library(ggrepel)
})


# ------------------------------------------------------------
# 2. Define input and output directories
# ------------------------------------------------------------

input_file <- file.path(
  "/scratch/[redacted]/RNA_seq_analysis",
  "featureCounts",
  "combined_counts_matrix.txt"
)

output_dir <- file.path(
  "/scratch/[redacted]/RNA_seq_analysis",
  "featureCounts",
  "PCA"
)

dir.create(
  output_dir,
  recursive = TRUE,
  showWarnings = FALSE
)


# ------------------------------------------------------------
# 3. Import featureCounts matrix
# ------------------------------------------------------------

if (!file.exists(input_file)) {
  stop("Input count matrix does not exist: ", input_file)
}

counts_df <- read.delim(
  input_file,
  header = TRUE,
  sep = "\t",
  stringsAsFactors = FALSE,
```

```
  check.names = FALSE,
  comment.char = "",
  quote = ""
)

if (ncol(counts_df) < 2L) {
  stop("The input matrix must contain a gene ID column and sample columns.")
}

# featureCounts output normally begins with:
# Geneid, Chr, Start, End, Strand, Length, followed by sample counts.
# The first column is the gene identifier.

gene_ids <- trimws(as.character(counts_df[[1]]))

if (anyNA(gene_ids) || any(gene_ids == "")) {
  stop("The gene ID column contains missing or empty identifiers.")
}

# Explicitly identify the 13 sample columns.
sample_names <- c(
  "1406",
  "25395",
  "14716",
  "29665",
  "22392",
  "18523",
  "10164",
  "13904",
  "20329",
  "21498",
  "30752",
  "3916",
  "4270"
)

missing_samples <- setdiff(sample_names, colnames(counts_df))

if (length(missing_samples) > 0L) {
  stop(
    "The following expected sample columns are missing: ",
    paste(missing_samples, collapse = ", "),
    "\nAvailable columns are: ",
    paste(colnames(counts_df), collapse = ", ")
  )
}

# Select only the required sample columns.
# Gene annotation columns such as Chr, Start, End, Strand and Length
# are deliberately excluded from the count matrix.

counts <- as.matrix(
  counts_df[, sample_names, drop = FALSE]
)
```

```
storage.mode(counts) <- "numeric"

if (anyNA(counts) || any(!is.finite(counts))) {
  stop("The count matrix contains missing or non-finite values.")
}

if (any(counts < 0)) {
  stop("The count matrix contains negative values.")
}

if (any(abs(counts - round(counts)) > 1e-6)) {
  stop(
    "The count matrix contains non-integer values. ",
    "Confirm that the input contains raw gene counts, not CPM, TPM or FPKM."
  )
}


# ------------------------------------------------------------
# 4. Merge duplicated gene IDs correctly
# ------------------------------------------------------------

unique_gene_ids <- unique(gene_ids)

counts_merged <- rowsum(
  counts,
  group = factor(gene_ids, levels = unique_gene_ids),
  reorder = FALSE
)

# Confirm that the merged matrix contains one row per gene ID.
if (anyDuplicated(rownames(counts_merged))) {
  stop("Duplicate gene IDs remain after merging.")
}

counts <- counts_merged

message("Number of input rows: ", length(gene_ids))
message("Number of unique gene IDs after merging: ", nrow(counts))


# ------------------------------------------------------------
# 5. Define sample metadata
# ------------------------------------------------------------

# Sample order must match the columns of the count matrix.

sample_metadata <- data.frame(
  sample = sample_names,
  rearrangement = c(
    "R",
    "NR",
    "NR",
    "NR",
    "NR",
```

```
    "NR",
    "NR",
    "NR",
    "R",
    "R",
    "R",
    "R",
    "R"
  ),
  stringsAsFactors = FALSE
)

rownames(sample_metadata) <- sample_metadata$sample

sample_metadata$rearrangement <- factor(
  sample_metadata$rearrangement,
  levels = c("R", "NR")
)

# Check sample alignment.
if (!identical(colnames(counts), rownames(sample_metadata))) {
  stop("Sample order in the count matrix does not match the metadata.")
}

if (nrow(sample_metadata) != 13L) {
  stop("Expected 13 samples, but the metadata contains ", nrow(sample_metadata), ".")
}

if (sum(sample_metadata$rearrangement == "R") != 6L ||
    sum(sample_metadata$rearrangement == "NR") != 7L) {
  stop("The metadata does not contain the expected 6 R and 7 NR samples.")
}


# ------------------------------------------------------------
# 6. Filter low-expression genes using CPM
# ------------------------------------------------------------

cpm_matrix <- edgeR::cpm(counts)

minimum_samples <- ceiling(ncol(counts) / 2)

keep_genes <- rowSums(cpm_matrix >= 1) >= minimum_samples

counts_filtered <- counts[keep_genes, , drop = FALSE]

message("Genes before CPM filtering: ", nrow(counts))
message("Genes after CPM filtering: ", nrow(counts_filtered))

if (nrow(counts_filtered) < 2L) {
  stop("Too few genes remain after CPM filtering to perform PCA.")
}


# ------------------------------------------------------------
```

```
# 7. Apply DESeq2 variance-stabilising transformation
# ------------------------------------------------------------

counts_filtered <- round(counts_filtered)

dds <- DESeqDataSetFromMatrix(
  countData = counts_filtered,
  colData = sample_metadata,
  design = ~ 1
)

# blind = TRUE performs the VST without using a group-specific
# design for estimating the transformation.
vsd <- vst(
  dds,
  blind = TRUE
)

vst_matrix <- assay(vsd)

# Export the complete VST matrix after CPM filtering.
write.table(
  data.frame(
    Geneid = rownames(vst_matrix),
    vst_matrix,
    check.names = FALSE
  ),
  file = file.path(output_dir, "vst_counts.tsv"),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)


# ------------------------------------------------------------
# 8. Select the 1,000 most variable genes
# ------------------------------------------------------------

# Calculate the variance of each gene across the 13 samples.

gene_variances <- matrixStats::rowVars(vst_matrix)

names(gene_variances) <- rownames(vst_matrix)


number_of_genes <- min(1000L, length(gene_variances))

top_1000_genes <- names(
  sort(
    gene_variances,
    decreasing = TRUE
  )
)[seq_len(number_of_genes)]

mat_top1000 <- vst_matrix[
```

```
  top_1000_genes,
  ,
  drop = FALSE
]

# Export the selected genes and their variances.
top_1000_gene_table <- data.frame(
  Geneid = top_1000_genes,
  Variance = gene_variances[top_1000_genes],
  stringsAsFactors = FALSE
)

write.table(
  top_1000_gene_table,
  file = file.path(output_dir, "top_1000_variable_genes.tsv"),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)

# Export the transformed expression values for the selected genes.
write.table(
  data.frame(
    Geneid = rownames(mat_top1000),
    mat_top1000,
    check.names = FALSE
  ),
  file = file.path(output_dir, "top_1000_vst_counts.tsv"),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)


# ------------------------------------------------------------
# 9. Perform principal component analysis
# ------------------------------------------------------------

pca_result <- prcomp(
  t(mat_top1000),
  scale. = FALSE,
  center = TRUE
)

# Calculate the percentage of variance explained by each PC.
variance_explained <- (
  pca_result$sdev^2 /
    sum(pca_result$sdev^2)
) * 100

variance_table <- data.frame(
  Principal_Component = paste0(
    "PC",
    seq_along(variance_explained)
  ),
```

```
  Variance_Explained_Percent = variance_explained
)

write.table(
  variance_table,
  file = file.path(output_dir, "PCA_variance_explained.tsv"),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)

# Export PCA coordinates, with metadata matched explicitly to the PCA sample order.

pca_coordinates <- data.frame(
  sample = rownames(pca_result$x),
  PC1 = pca_result$x[, 1],
  PC2 = pca_result$x[, 2],
  rearrangement = sample_metadata[
    rownames(pca_result$x),
    "rearrangement"
  ],
  stringsAsFactors = FALSE
)

write.table(
  pca_coordinates,
  file = file.path(output_dir, "PCA_coordinates.tsv"),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)


# ------------------------------------------------------------
# 10. Generate PCA plot
# ------------------------------------------------------------

pc1_percent <- round(variance_explained[1], 1)
pc2_percent <- round(variance_explained[2], 1)

pca_plot <- ggplot(
  pca_coordinates,
  aes(
    x = PC1,
    y = PC2,
    colour = rearrangement,
    label = sample
  )
) +
  geom_point(
    size = 4,
    alpha = 0.9
  ) +
  ggrepel::geom_text_repel(
    size = 3.5,
```

```
    max.overlaps = Inf
  ) +
  scale_colour_manual(
    values = c(
      "R" = "firebrick3",
      "NR" = "navy"
    ),
    breaks = c("R", "NR"),
    labels = c("R", "NR"),
    name = "Rearrangement"
  ) +
  labs(
    title = "PCA of DLBCL RNA-seq Samples",
    x = paste0("PC1 (", pc1_percent, "% variance)"),
    y = paste0("PC2 (", pc2_percent, "% variance)")
  ) +
  theme_classic(base_size = 13) +
  theme(
    plot.title = element_text(
      hjust = 0.5,
      face = "bold"
    ),
    legend.position = "right"
  )

# Save a high-resolution PNG.
ggsave(
  filename = file.path(output_dir, "PCA_plot.png"),
  plot = pca_plot,
  width = 9,
  height = 7,
  dpi = 300
)

# Also save a PDF version.
ggsave(
  filename = file.path(output_dir, "PCA_plot.pdf"),
  plot = pca_plot,
  width = 9,
  height = 7
)


# ------------------------------------------------------------
# 11. Save the PCA object and analysis metadata
# ------------------------------------------------------------

saveRDS(
  pca_result,
  file = file.path(output_dir, "PCA_result.rds")
)

write.table(
  sample_metadata,
  file = file.path(output_dir, "PCA_sample_metadata.tsv"),
```

```
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)


# ------------------------------------------------------------
# 12. Report completion
# ------------------------------------------------------------

message("PCA analysis completed successfully.")
message("Input matrix: ", input_file)
message("Output directory: ", output_dir)
message("Samples analysed: ", ncol(counts_filtered))
message("Genes after CPM filtering: ", nrow(vst_matrix))
message("Genes selected for PCA: ", nrow(mat_top1000))
message(
  "Variance explained by PC1: ",
  round(variance_explained[1], 2),
  "%"
)
message(
  "Variance explained by PC2: ",
  round(variance_explained[2], 2),
  "%"
)
```

DESeq2

BASH script:

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --job-name=DESeq2_R
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL
```

```
set -e
set -o pipefail

echo "Job started on $(hostname)"

# Load miniconda (to get conda command)
export PATH="/opt/exp_soft/miniconda3-py3.12/bin:$PATH"
source /opt/exp_soft/miniconda3-py3.12/etc/profile.d/conda.sh

echo "Activating R conda environment..."
conda activate /scratch/[redacted]/conda_envs/conda_env

# Sanity check
echo "Which R am I using?"
which R
R --version

# Run the R script
echo "Running DESeq2_final.R..."
Rscript DESeq2_final.R

echo "Job finished"
```

R script:

```
#!/usr/bin/env Rscript

suppressPackageStartupMessages({
  library(DESeq2)
  library(tidyverse)
  library(ggplot2)
  library(pheatmap)
  library(matrixStats)
  library(ggrepel)
  library(edgeR)
  library(apeglm)
})
```

```
# ----------------------------
# Directories
# ----------------------------

input_dir <- "/scratch/[redacted]/RNA_seq_analysis/featureCounts"
output_dir <- "/scratch/[redacted]/RNA_seq_analysis/DESeq2"

counts_file <- file.path(
  input_dir,
  "combined_counts_matrix.txt"
)

if (!dir.exists(output_dir)) {
  dir.create(output_dir, recursive = TRUE)
}

message("Reading counts from: ", counts_file)

if (!file.exists(counts_file)) {
  stop("Count file not found: ", counts_file)
}

# ----------------------------
# Load counts
# ----------------------------

counts_df <- read.delim(
  counts_file,
  header = TRUE,
  sep = "\t",
  stringsAsFactors = FALSE,
  check.names = FALSE,
  comment.char = "",
  quote = ""
)

if (ncol(counts_df) < 2) {
  stop(
    "The count matrix must contain a gene ID column ",
    "and at least one sample column."
  )
}

# The first column must contain the original gene IDs.
gene_ids <- as.character(counts_df[[1]])

if (anyNA(gene_ids) || any(trimws(gene_ids) == "")) {
  stop(
    "The gene ID column contains missing or empty identifiers. ",
    "Please inspect the featureCounts matrix."
  )
}

# Select the 13 intended sample columns explicitly.
samples_to_keep <- c(
```

```
  "1406", "25395", "14716", "29665", "22392", "18523",
  "10164", "13904", "20329", "21498", "30752", "3916", "4270"
)

missing_samples <- setdiff(
  samples_to_keep,
  colnames(counts_df)
)

if (length(missing_samples) > 0) {
  stop(
    "These samples are missing from the count matrix: ",
    paste(missing_samples, collapse = ", ")
  )
}

counts_mat <- as.matrix(
  counts_df[, samples_to_keep, drop = FALSE]
)

# Confirm that all selected sample columns contain numeric counts.
suppressWarnings(
  storage.mode(counts_mat) <- "numeric"
)

if (anyNA(counts_mat) || any(!is.finite(counts_mat))) {
  stop(
    "The selected sample columns contain missing or non-numeric ",
    "values. Check the input matrix."
  )
}

if (any(counts_mat < 0)) {
  stop("Negative counts were detected.")
}

if (any(abs(counts_mat - round(counts_mat)) > 1e-8)) {
  stop(
    "Non-integer counts were detected. DESeq2 requires ",
    "non-negative integer count data."
  )
}

storage.mode(counts_mat) <- "integer"

rownames(counts_mat) <- gene_ids

message(
  "Original matrix dimensions: ",
  nrow(counts_mat), " genes x ",
  ncol(counts_mat), " samples"
)

message(
  "Samples retained: ",
```

```
  paste(colnames(counts_mat), collapse = ", ")
)

# ----------------------------
# Sample metadata
# ----------------------------

sample_rearrangement <- c(
  "R", "NR", "NR", "NR", "NR", "NR", "NR", "NR",
  "R", "R", "R", "R", "R"
)

sample_hiv_status <- c(
  "N", "N", "N", "P", "N",
  "N", "N", "N",
  "N", "N", "N", "N", "N"
)

sample_gender <- c(
  "M", "M", "M", "F", "M", "F", "M",
  "F", "M", "F", "F", "M", "F"
)

coldata <- data.frame(
  row.names = samples_to_keep,
  rearrangement = factor(
    sample_rearrangement,
    levels = c("NR", "R")
  ),
  HIV_status = factor(sample_hiv_status),
  gender = factor(sample_gender)
)

# Confirm that metadata and count-matrix sample ordering agree.
if (!identical(colnames(counts_mat), rownames(coldata))) {
  stop(
    "The count matrix sample order does not match ",
    "the metadata row order."
  )
}

message("\nSample metadata:")
print(coldata)

# ----------------------------
# Merge duplicate original gene IDs
# ----------------------------

message("\nChecking duplicate gene IDs...")

n_original_rows <- length(gene_ids)
n_unique_ids <- length(unique(gene_ids))
n_duplicate_rows <- n_original_rows - n_unique_ids

message("Original gene rows: ", n_original_rows)
```

```
message("Unique original gene IDs: ", n_unique_ids)
message("Duplicate rows to be merged: ", n_duplicate_rows)

if (n_duplicate_rows > 0) {

  message("Merging duplicate gene IDs by summing their counts...")

  # rowsum() retains the original gene IDs as row names.
  # reorder = FALSE preserves the order of first occurrence.
  counts_mat <- rowsum(
    counts_mat,
    group = factor(
      gene_ids,
      levels = unique(gene_ids)
    ),
    reorder = FALSE
  )

}

if (anyDuplicated(rownames(counts_mat))) {
  stop(
    "Duplicate gene IDs remain after merging. ",
    "Please inspect the count matrix."
  )
}

message(
  "Dimensions after duplicate merging: ",
  nrow(counts_mat), " genes x ",
  ncol(counts_mat), " samples"
)

# ----------------------------
# CPM filtering
# ----------------------------

message("\nApplying CPM filtering...")

dge <- DGEList(counts = counts_mat)

cpm_mat <- cpm(dge)

min_samples <- ceiling(ncol(counts_mat) / 2)

keep <- rowSums(cpm_mat >= 1) >= min_samples

counts_mat <- counts_mat[keep, , drop = FALSE]

message(
  "CPM threshold: >= 1 in at least ",
  min_samples, " samples"
)

message(
```

```
  "Dimensions after CPM filtering: ",
  nrow(counts_mat), " genes x ",
  ncol(counts_mat), " samples"
)

if (nrow(counts_mat) == 0) {
  stop("No genes remain after CPM filtering.")
}

if (nrow(counts_mat) < 1000) {
  warning(
    "Fewer than 1,000 genes remain after filtering. ",
    "The PCA and heatmap code will use the available genes."
  )
}

# ------------------------------
# DESeq2 differential expression analysis
# ------------------------------

message("\nRunning DESeq2...")

dds <- DESeqDataSetFromMatrix(
  countData = counts_mat,
  colData = coldata,
  design = ~ rearrangement
)

dds <- DESeq(dds)

res <- results(
  dds,
  contrast = c("rearrangement", "R", "NR")
)

# ------------------------------
# LFC shrinkage
# ------------------------------

res_shrink <- lfcShrink(
  dds,
  coef = "rearrangement_R_vs_NR",
  type = "apeglm"
)

res_df_shrink <- as.data.frame(res_shrink) %>%
  rownames_to_column("gene") %>%
  mutate(
    padj = ifelse(is.na(padj), 1, padj),
    significant = padj < 0.05 &
      abs(log2FoldChange) > 1
  )

# ------------------------------
# DEG summary
```

```
# ----------------------------

significant_degs <- res_df_shrink %>%
  filter(significant)

upregulated_degs <- significant_degs %>%
  filter(log2FoldChange > 1)

downregulated_degs <- significant_degs %>%
  filter(log2FoldChange < -1)

message("")
message("========================================")
message("Differentially Expressed Gene Summary")
message("========================================")
message("Significance criteria: padj < 0.05 and |log2FC| > 1")
message("Total significant DEGs: ", nrow(significant_degs))
message(
  "Significantly upregulated in R vs NR: ",
  nrow(upregulated_degs)
)
message(
  "Significantly downregulated in R vs NR: ",
  nrow(downregulated_degs)
)
message("========================================")
message("")

# ----------------------------
# Results table
# ----------------------------

res_df <- as.data.frame(res) %>%
  rownames_to_column("gene") %>%
  mutate(
    padj = ifelse(is.na(padj), 1, padj),
    significant = padj < 0.05 &
      abs(log2FoldChange) > 1
  )

write.table(
  res_df,
  file = file.path(output_dir, "DESeq2_results.tsv"),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)

# ----------------------------
# VST
# ----------------------------

message("\nApplying variance-stabilising transformation...")

vsd <- vst(
```

```
  dds,
  blind = FALSE
)

vst_mat <- assay(vsd)

# ----------------------------
# 1. PCA
# ----------------------------

message("\nGenerating PCA...")

gene_variances <- rowVars(vst_mat)

number_of_pca_genes <- min(
  1000L,
  length(gene_variances)
)

top_1000_genes <- order(
  gene_variances,
  decreasing = TRUE
)[seq_len(number_of_pca_genes)]

mat_top1000 <- vst_mat[
  top_1000_genes,
  ,
  drop = FALSE
]

pca_matrix <- t(mat_top1000)

pca_result <- prcomp(
  pca_matrix,
  scale. = FALSE,
  center = TRUE
)

percentVar <- (
  pca_result$sdev^2 /
    sum(pca_result$sdev^2)
) * 100

pca_res <- data.frame(
  PC1 = pca_result$x[, 1],
  PC2 = pca_result$x[, 2],
  rearrangement = coldata$rearrangement,
  sample = rownames(pca_result$x)
)

pca_plot <- ggplot(
  pca_res,
  aes(
    x = PC1,
    y = PC2,
```

```
    color = rearrangement,
    label = sample
  )
) +
  geom_point(size = 3) +
  geom_text_repel(size = 3) +
  scale_color_manual(
    values = c(
      NR = "navy",
      R = "firebrick3"
    )
  ) +
  xlab(
    paste0(
      "PC1: ",
      round(percentVar[1], 1),
      "% variance"
    )
  ) +
  ylab(
    paste0(
      "PC2: ",
      round(percentVar[2], 1),
      "% variance"
    )
  ) +
  theme_bw() +
  ggtitle("PCA of samples")

ggsave(
  file.path(output_dir, "PCA_plot.png"),
  pca_plot,
  width = 8,
  height = 6,
  dpi = 300
)

# Save PCA coordinates
write.table(
  pca_res,
  file = file.path(output_dir, "PCA_coordinates.tsv"),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)

# Save the genes used for PCA
write.table(
  data.frame(
    gene = rownames(mat_top1000),
    variance = gene_variances[top_1000_genes]
  ),
  file = file.path(output_dir, "PCA_top1000_genes.tsv"),
  sep = "\t",
  quote = FALSE,
```

```
  row.names = FALSE
)

message(
  "PCA generated using ",
  number_of_pca_genes,
  " most variable genes."
)

# ------------------------------
# 2. Dispersion plot
# ------------------------------

png(
  file.path(output_dir, "Dispersion_plot.png"),
  width = 800,
  height = 600
)

plotDispEsts(dds)

dev.off()

# ------------------------------
# 3. MA plot (shrunken LFC)
# ------------------------------

ma_plot <- ggplot(
  res_df_shrink,
  aes(
    x = baseMean,
    y = log2FoldChange
  )
) +
  geom_point(alpha = 0.4) +
  geom_point(
    data = subset(
      res_df_shrink,
      significant & log2FoldChange > 0
    ),
    color = "red",
    alpha = 0.8
  ) +
  geom_point(
    data = subset(
      res_df_shrink,
      significant & log2FoldChange < 0
    ),
    color = "green",
    alpha = 0.8
  ) +
  scale_x_log10() +
  geom_hline(
    yintercept = 0,
    linetype = "dashed"
```

```
  ) +
  theme_bw() +
  ggtitle("MA Plot (shrunken LFC)")

ggsave(
  file.path(output_dir, "MA_plot.png"),
  ma_plot,
  width = 8,
  height = 6,
  dpi = 300
)

# ----------------------------------
# 4. Volcano plot (unshrunken L2FCs)
# ----------------------------------

sig_res <- res_df %>%
  filter(
    padj < 0.05 &
      abs(log2FoldChange) > 1
  ) %>%
  mutate(
    score = -log10(padj) * abs(log2FoldChange)
  )

top_overexpressed <- sig_res %>%
  filter(log2FoldChange > 0) %>%
  arrange(desc(score)) %>%
  slice_head(n = 5)

top_underexpressed <- sig_res %>%
  filter(log2FoldChange < 0) %>%
  arrange(desc(score)) %>%
  slice_head(n = 5)

top_genes <- bind_rows(
  top_overexpressed,
  top_underexpressed
)

volcano_plot <- ggplot(
  res_df,
  aes(
    x = log2FoldChange,
    y = -log10(padj)
  )
) +
  geom_point(alpha = 0.4) +
  geom_point(
    data = subset(
      res_df,
      significant & log2FoldChange > 0
    ),
    color = "red",
    alpha = 0.8
```

```
  ) +
  geom_point(
    data = subset(
      res_df,
      significant & log2FoldChange < 0
    ),
    color = "green",
    alpha = 0.8
  ) +
  geom_text_repel(
    data = top_genes,
    aes(label = gene),
    color = "black",
    size = 4,
    max.overlaps = Inf
  ) +
  theme_bw() +
  ggtitle("Volcano Plot: Top 5 Over- & Under-expressed Genes") +
  xlab("Log2 Fold Change") +
  ylab("-log10 adjusted p-value")

ggsave(
  file.path(
    output_dir,
    "Volcano_plot_top5_over_under.png"
  ),
  volcano_plot,
  width = 8,
  height = 6,
  dpi = 300
)

# ----------------------------
# Statistics table for top 5
# upregulated and downregulated genes
# ----------------------------

top_genes_stats <- bind_rows(
  top_overexpressed %>%
    mutate(direction = "Upregulated") %>%
    select(
      gene,
      direction,
      log2FoldChange,
      pvalue,
      padj
    ),
  top_underexpressed %>%
    mutate(direction = "Downregulated") %>%
    select(
      gene,
      direction,
      log2FoldChange,
      pvalue,
      padj
```

```
    )
) %>%
  rename(`p-value` = pvalue) %>%
  mutate(
    direction = factor(
      direction,
      levels = c(
        "Upregulated",
        "Downregulated"
      )
    )
  ) %>%
  arrange(
    direction,
    desc(abs(log2FoldChange))
  )

write.table(
  top_genes_stats,
  file = file.path(
    output_dir,
    "Top5_upregulated_downregulated_genes_statistics.tsv"
  ),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)

# ------------------------------
# 5. Heatmap
# ------------------------------

message("\nGenerating heatmap...")

mat_top1000_scaled <- t(
  scale(t(mat_top1000))
)

# Remove genes with incomplete scaled values, such as genes
# with zero variance across all samples.
mat_top1000_scaled <- mat_top1000_scaled[
  complete.cases(mat_top1000_scaled),
  ,
  drop = FALSE
]

if (nrow(mat_top1000_scaled) == 0) {
  stop(
    "No genes remain for the heatmap after scaling."
  )
}

# Cluster samples using all available scaled top-1,000 genes.
sample_dist <- dist(
  t(mat_top1000_scaled)
```

```
)

sample_hclust <- hclust(sample_dist)

# Select up to the top 50 most variable genes from the
# original top-1,000-gene set.
heatmap_variances <- rowVars(
  mat_top1000
)

number_of_heatmap_genes <- min(
  50L,
  length(heatmap_variances)
)

top_50_genes <- order(
  heatmap_variances,
  decreasing = TRUE
)[seq_len(number_of_heatmap_genes)]

top_50_gene_ids <- rownames(mat_top1000)[
  top_50_genes
]

mat_top50 <- mat_top1000_scaled[
  top_50_gene_ids,
  ,
  drop = FALSE
]

# Change only the displayed annotation label from gender to sex.
annotation_col <- coldata

colnames(annotation_col)[
  colnames(annotation_col) == "gender"
] <- "sex"

annotation_colors <- list(
  rearrangement = c(
    NR = "blue",
    R = "red"
  ),
  HIV_status = c(
    N = "green",
    P = "pink"
  ),
  sex = c(
    F = "purple",
    M = "yellow"
  )
)

png(
  file.path(
    output_dir,
```

```
    "Top50_variable_genes_heatmap.png"
  ),
  width = 3000,
  height = 4000,
  res = 300
)

pheatmap(
  mat_top50,
  annotation_col = annotation_col,
  annotation_colors = annotation_colors,
  show_rownames = TRUE,
  fontsize_row = 10,
  fontsize_col = 10,
  cluster_rows = TRUE,
  cluster_cols = sample_hclust,
  main = "Top 50 Most Variable Genes"
)

dev.off()

# ------------------------------
# Save VST counts
# ------------------------------

write.table(
  vst_mat,
  file = file.path(
    output_dir,
    "vst_counts.tsv"
  ),
  sep = "\t",
  quote = FALSE,
  col.names = NA
)

message(
  "\nAll analyses complete. Outputs saved to: ",
  output_dir
)
```

Fgsea

BASH script:

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --job-name=GSEA
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

set -e
set -o pipefail

echo "Job started on $(hostname)"

# Load miniconda (to get conda command)
export PATH="/opt/exp_soft/miniconda3-py3.12/bin:$PATH"
source /opt/exp_soft/miniconda3-py3.12/etc/profile.d/conda.sh

echo "Activating R conda environment..."
conda activate /scratch/[redacted]/conda_envs/conda_env

# Sanity check
echo "Which R am I using?"
which R
R --version

# Run the R script
echo "Running fgsea_final.R..."
Rscript fgsea_final.R

echo "Job finished"
```

R script:

```
#!/usr/bin/env Rscript

# ==============================
# GSEA using fgsea
# ==============================

suppressPackageStartupMessages({
  library(data.table)
  library(fgsea)
  library(ggplot2)
  library(pheatmap)
})

# ----------------------------
# Version information
# ----------------------------
cat("\n==============================\n")
cat("SESSION INFO (START)\n")
cat("==============================\n\n")

cat("R version:\n")
cat(R.version.string, "\n\n")
cat("Platform:\n")
cat(R.version$platform, "\n\n")

pkgs <- c("data.table", "fgsea", "ggplot2", "pheatmap")
cat("Package versions:\n")
for (pkg in pkgs) {
  if (requireNamespace(pkg, quietly = TRUE)) {
    cat(pkg, ":", as.character(packageVersion(pkg)), "\n")
  } else {
    cat(pkg, ": NOT INSTALLED\n")
  }
}

if (requireNamespace("BiocManager", quietly = TRUE)) {
  cat("\nBioconductor version:\n")
  cat(as.character(BiocManager::version()), "\n")
}

cat("\n==============================\n\n")

# ----------------------------
# Input files
# ----------------------------
deseq_file <- "/scratch/[redacted]/RNA_seq_analysis/DESeq2/DESeq2_results.tsv"
hallmark_gmt <-
"/scratch/[redacted]/RNA_seq_analysis/GSEA/gmt_files/h.all.v2026.1.Hs.symbols.gmt.txt"

# ----------------------------
# Load DESeq2 results
# ----------------------------
res <- fread(deseq_file)
```

```
res <- res[!is.na(log2FoldChange) & !is.na(padj) & !is.na(pvalue)]
res <- res[!duplicated(gene)]
res$gene <- toupper(res$gene)

# ----------------------------
# Create DESeq2 Wald statistic
# ----------------------------
res$stat <- res$log2FoldChange / res$lfcSE

# ----------------------------
# Create ranked gene list
# ----------------------------
ranks <- res$stat
names(ranks) <- res$gene
ranks <- sort(ranks, decreasing = TRUE)

cat("Total genes used for GSEA:", length(ranks), "\n")

# ----------------------------
# Load Hallmark GMT gene set
# ----------------------------
hallmark <- gmtPathways(hallmark_gmt)

# ----------------------------
# Hallmark gene-set coverage
# ----------------------------
hallmark_genes <- unique(unlist(hallmark))

hallmark_overlap <- intersect(names(ranks), hallmark_genes)

cat("\n==============================\n")
cat("HALLMARK GENE-SET COVERAGE\n")
cat("==============================\n")
cat("Total genes used for GSEA:", length(ranks), "\n")
cat("Total unique genes in Hallmark gene sets:", length(hallmark_genes), "\n")
cat("Hallmark genes present in ranked list:", length(hallmark_overlap), "\n")
cat("Coverage of ranked GSEA genes:",
    round(100 * length(hallmark_overlap) / length(ranks), 2), "%\n")
cat("Coverage of Hallmark gene set:",
    round(100 * length(hallmark_overlap) / length(hallmark_genes), 2), "%\n")

# ----------------------------
# Output directory
# ----------------------------
output_dir <- "/scratch/[redacted]/RNA_seq_analysis/GSEA"
dir.create(output_dir, recursive = TRUE, showWarnings = FALSE)

# ----------------------------
# FGSEA runner
# ----------------------------
run_fgsea <- function(ranked_genes, label){

  cat("\n==============================\n")
  cat("Running FGSEA:", label, "\n")
```

```
if(length(ranked_genes) < 100){
  cat("Too few genes, skipping\n")
  return(NULL)
}

if(length(unique(ranked_genes)) < 10){
  cat("Too little variation, skipping\n")
  return(NULL)
}

run_one_db <- function(pathways, db_name){

  cat("Running", db_name, "...\n")

  fg <- fgsea(
    pathways = pathways,
    stats    = ranked_genes,
    minSize  = 15,
    maxSize  = 500,
    eps      = 0
  )

  fg <- fg[order(padj), ]

  out_dir <- file.path(output_dir, label, db_name)
  dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)

  fwrite(
    fg,
    file.path(out_dir, "fgsea_results_all.tsv"),
    sep = "\t"
  )

  fg_sig <- fg[padj < 0.05]

  if(nrow(fg_sig) == 0){
    cat("No significant pathways\n")
    return(NULL)
  }

  fg_up   <- fg_sig[NES > 0]
  fg_down <- fg_sig[NES < 0]

  fwrite(
    fg_up,
    file.path(out_dir, "fgsea_upregulated.tsv"),
    sep = "\t"
  )

  fwrite(
    fg_down,
    file.path(out_dir, "fgsea_downregulated.tsv"),
    sep = "\t"
  )
```

```
    cat(
      "Significant pathways:",
      "\n  Upregulated:", nrow(fg_up),
      "\n  Downregulated:", nrow(fg_down), "\n"
    )

    top_up   <- head(fg_up[order(-NES)], 10)
    top_down <- head(fg_down[order(NES)], 10)

    pdf(
      file.path(out_dir, "enrichment_plots_split.pdf"),
      width = 8,
      height = 6
    )

    for(p in c(top_up$pathway, top_down$pathway)){

      if(p %in% names(pathways)){

        print(
          plotEnrichment(
            pathways[[p]],
            ranked_genes
          ) +
            ggtitle(
              paste(label, db_name, p)
            )
        )
      }
    }

    dev.off()

  }

  # Run on Hallmark database only
  run_one_db(
    hallmark,
    "Hallmark"
  )

}

# ------------------------------
# RUN FGSEA
# ------------------------------
run_fgsea(
  ranks,
  "All_genes"
)

cat(
  "\n FGSEA completed successfully.\n"
)
```

Gene lists

BASH script:

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --job-name=gene_lists
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

set -e
set -o pipefail

# Activate environment (assuming it exists)
export PATH="/opt/exp_soft/miniconda3-py3.12/bin:$PATH"
source /opt/exp_soft/miniconda3-py3.12/etc/profile.d/conda.sh
echo "Activating environment..."
conda activate /scratch/[redacted]/conda_envs/conda_env

# Run Python script
echo "Running over_and_underexpressed_final.py..."
python over_and_underexpressed_final.py
```

Python script:

```
#---------------------------------
# Packages and directories
#---------------------------------
import os
import pandas as pd

# Base directory
base_dir = "/scratch/[redacted]/RNA_seq_analysis/DESeq2"
os.makedirs(base_dir, exist_ok=True)

# Read DESeq2 results
df = pd.read_csv(
    os.path.join(base_dir, "DESeq2_results.tsv"),
    sep="\t"
)

# Remove rows with missing statistics
df = df.dropna(subset=["padj", "log2FoldChange", "gene"])

# ------------------------------
# Significant DEGs (padj only)
# ------------------------------
sig_df = df[df["padj"] < 0.05]

sig_df["gene"].to_csv(
    os.path.join(base_dir, "significant_DEGs.txt"),
    index=False,
    header=False
)

# ------------------------------
# Overexpressed genes
# ------------------------------
over_genes = sig_df.loc[
    sig_df["log2FoldChange"] > 1,
    "gene"
]

over_genes.to_csv(
    os.path.join(base_dir, "overexpressed_genes.txt"),
    index=False,
    header=False
)

print("Overexpressed gene list generated!")

# ------------------------------
# Underexpressed genes
# ------------------------------
under_genes = sig_df.loc[
    sig_df["log2FoldChange"] < -1,
    "gene"
]
```

```
under_genes.to_csv(
    os.path.join(base_dir, "underexpressed_genes.txt"),
    index=False,
    header=False
)

print("Underexpressed gene list generated!")

# ----------------------------
# Checks
# ----------------------------
print(f"Overexpressed genes: {over_genes.shape[0]}")
print(f"Underexpressed genes: {under_genes.shape[0]}")

# ----------------------------
# Generate ranked gene list
# ----------------------------
ranked_df = df[["gene", "log2FoldChange"]].copy()
ranked_df = ranked_df.sort_values(
    by="log2FoldChange",
    ascending=False
)

ranked_df.to_csv(
    os.path.join(base_dir, "ranked_genes_log2FC.rnk"),
    sep="\t",
    index=False,
    header=False
)

print("Ranked gene list generated!")
```

ClusterProfiler

BASH script:

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --job-name=GO_analysis
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

set -e
set -o pipefail

echo "Job started on $(hostname)"

# Load miniconda (to get conda command)
export PATH="/opt/exp_soft/miniconda3-py3.12/bin:$PATH"
source /opt/exp_soft/miniconda3-py3.12/etc/profile.d/conda.sh

echo "Activating R conda environment..."
conda activate /scratch/[redacted]/conda_envs/conda_env

# Sanity check
echo "Which R am I using?"
which R
R --version

# Run the R script
echo "Running GO_analysis_final.R..."
Rscript GO_analysis_final.R

echo "Job finished"
```

R script:

```
#!/usr/bin/env Rscript

# ==============================
# GO ORA using clusterProfiler with visualization
# ==============================

suppressPackageStartupMessages({
  library(clusterProfiler)
  library(org.Hs.eg.db)
  library(dplyr)
  library(readr)
  library(stringr)
  library(ggplot2)
})

# ----------------------------
# Input gene lists
# ----------------------------
over_file <- "/scratch/[redacted]/RNA_seq_analysis/DESeq2/overexpressed_genes.txt"
under_file <- "/scratch/[redacted]/RNA_seq_analysis/DESeq2/underexpressed_genes.txt"

over_genes <- read_lines(over_file) %>% na.omit()
under_genes <- read_lines(under_file) %>% na.omit()

cat("Overexpressed genes:", length(over_genes), "\n")
cat("Underexpressed genes:", length(under_genes), "\n")

# ----------------------------
# Output directory
# ----------------------------
base_outdir <- "/scratch/[redacted]/RNA_seq_analysis/GO"
dir.create(base_outdir, recursive = TRUE, showWarnings = FALSE)

# ----------------------------------
# Convert gene symbols to Entrez IDs
# ----------------------------------
convert_to_entrez <- function(genes) {
  entrez <- bitr(
    genes,
    fromType = "SYMBOL",
    toType = "ENTREZID",
    OrgDb = org.Hs.eg.db
  )

  if (nrow(entrez) == 0) {
    warning("No genes could be mapped to Entrez IDs.")
    return(character(0))
  }

  return(unique(entrez$ENTREZID))
}

# ------------------------------------------
```

```
# Function to run GO ORA and generate dot plots
# ------------------------------------------------
run_go_ora <- function(genes, label) {

  if (length(genes) < 5) {
    cat(sprintf("Skipping %s (too few genes)\n", label))
    return(NULL)
  }

  cat(sprintf("\nRunning GO ORA for %s genes...\n", label))

  entrez_genes <- convert_to_entrez(genes)

  if (length(entrez_genes) < 5) {
    cat(sprintf(
      "Too few genes mapped to Entrez IDs for %s. Skipping.\n",
      label
    ))
    return(NULL)
  }

  go_dirs <- c(
    "BP" = "Biological_Process",
    "MF" = "Molecular_Function",
    "CC" = "Cellular_Component"
  )

  for (go_type in names(go_dirs)) {

    cat(sprintf("Running GO-%s for %s...\n", go_type, label))

    go_res <- enrichGO(
      gene          = entrez_genes,
      OrgDb         = org.Hs.eg.db,
      keyType       = "ENTREZID",
      ont           = go_type,
      pAdjustMethod = "BH",
      pvalueCutoff  = 0.05,
      qvalueCutoff  = 0.05,
      readable      = TRUE
    )

    outdir <- file.path(
      base_outdir,
      label,
      paste0("GO_", go_type)
    )

    dir.create(outdir, recursive = TRUE, showWarnings = FALSE)

    # ----------------------------
    # Save results
    # ----------------------------
    write.csv(
      as.data.frame(go_res),
```

```
      file = file.path(
        outdir,
        paste0(label, "_GO_", go_type, "_ORA.csv")
      ),
      row.names = FALSE
    )

    cat("Saved GO-", go_type, "results to", outdir, "\n")

    # ------------------------------
    # Dot plot
    # ------------------------------
    if (nrow(go_res) > 0) {

      dot_file <- file.path(
        outdir,
        paste0(label, "_GO_", go_type, "_dotplot.pdf")
      )

      pdf(dot_file, width = 8, height = 6)

      print(
        dotplot(go_res, showCategory = 20) +
          ggtitle(paste(label, "GO", go_type)) +
          theme(axis.text.y = element_text(size = 6))
      )

      dev.off()

    } else {

      cat(sprintf(
        "No significant GO-%s terms for %s. Skipping dot plot.\n",
        go_type,
        label
      ))
    }
  }
}

# ------------------------------
# Run GO ORA
# ------------------------------
run_go_ora(over_genes, "Overexpressed")
run_go_ora(under_genes, "Underexpressed")

cat("\nGO over-representation analysis completed successfully!\n")
```

TPM format matrix

BASH script:

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --job-name=cleaning_cibersortx
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

set -e
set -o pipefail

echo "Job started on $(hostname)"

# Load miniconda (to get conda command)
export PATH="/opt/exp_soft/miniconda3-py3.12/bin:$PATH"
source /opt/exp_soft/miniconda3-py3.12/etc/profile.d/conda.sh

echo "Activating R conda environment..."
conda activate /scratch/[redacted]/conda_envs/conda_env

# Sanity check
echo "Which R am I using?"
which R
R --version

# Run the R script
echo "Running CIBERSORTx_final.R..."
Rscript CIBERSORTx_final.R

echo "Job finished"
```

R script:

```
#!/usr/bin/env Rscript

suppressPackageStartupMessages({
  library(data.table)
})

# ----------------------------
# Input / output
# ----------------------------
counts_file <-
"/scratch/[redacted]/RNA_seq_analysis/featureCounts/combined_featureCounts.txt"

output_dir <- "/scratch/[redacted]/RNA_seq_analysis/CIBERSORTx"

clean_counts_file <- file.path(
  output_dir,
  "cleaned_featureCounts_raw_counts.tsv"
)

tpm_file <- file.path(
  output_dir,
  "TPM_matrix.tsv"
)

dir.create(
  output_dir,
  recursive = TRUE,
  showWarnings = FALSE
)

# ----------------------------
# Read original featureCounts
# ----------------------------
counts <- fread(
  counts_file,
  header = TRUE,
  sep = "\t",
  check.names = FALSE
)

cat("Original dimensions:",
    nrow(counts), "genes x",
    ncol(counts), "columns\n")

# ----------------------------
# Rename first column
# ----------------------------
setnames(counts, 1, "Geneid")

# ----------------------------
# Identify featureCounts columns
# ----------------------------
```

```
annotation_cols <- c(
  "Geneid",
  "Chr",
  "Start",
  "End",
  "Strand",
  "Length"
)

# Check that all expected annotation columns are present
missing_cols <- setdiff(annotation_cols, colnames(counts))

if (length(missing_cols) > 0) {
  stop(
    "The following expected featureCounts columns are missing: ",
    paste(missing_cols, collapse = ", ")
  )
}

# ------------------------------
# Identify sample columns
# ------------------------------
sample_cols_original <- setdiff(
  colnames(counts),
  annotation_cols
)

cat("\nOriginal sample columns:\n")
print(sample_cols_original)

# ------------------------------
# Shorten sample names
# ------------------------------

short_sample_names <- sub(
  "^.*/",
  "",
  sample_cols_original
)

short_sample_names <- sub(
  "_Aligned\\.sortedByCoord\\.out\\.bam$",
  "",
  short_sample_names
)

# Extract the actual sample ID
short_sample_names <- ifelse(
  grepl("-", short_sample_names),
  sub("^[^-]*-", "", short_sample_names),
  sub("_.*$", "", short_sample_names)
)

# Check mapping
sample_mapping <- data.table(
```

```
  original = sample_cols_original,
  shortened = short_sample_names
)

cat("\nSample-name mapping:\n")
print(sample_mapping)

# Check for duplicate shortened names
if (anyDuplicated(short_sample_names)) {
  stop(
    "Duplicate shortened sample names detected: ",
    paste(
      unique(short_sample_names[duplicated(short_sample_names)]),
      collapse = ", "
    )
  )
}

# ----------------------------
# Rename sample columns
# ----------------------------
setnames(
  counts,
  sample_cols_original,
  short_sample_names
)

# ----------------------------
# Define final sample columns
# ----------------------------
sample_cols <- setdiff(
  colnames(counts),
  annotation_cols
)

cat("\nFinal samples:\n")
print(sample_cols)

cat("\nNumber of samples:", length(sample_cols), "\n")

# ----------------------------
# Clean Gene IDs
# ----------------------------
counts[, Geneid := trimws(Geneid)]

# ----------------------------
# Check gene IDs
# ----------------------------
if (any(is.na(counts$Geneid)) ||
    any(counts$Geneid == "")) {
  stop("Missing or empty GeneID detected.")
}

# ----------------------------
# Check gene lengths
```

```
# ----------------------------
if (any(is.na(counts$Length))) {
  stop("Missing gene lengths detected.")
}

if (any(counts$Length <= 0)) {
  stop("Zero or negative gene lengths detected.")
}

# ----------------------------
# Check for duplicate GeneIDs
# ----------------------------
if (anyDuplicated(counts$Geneid)) {

  cat(
    "\nWARNING: Duplicate GeneIDs detected.\n"
  )

  cat(
    "Number of duplicated GeneIDs:",
    sum(duplicated(counts$Geneid)),
    "\n"
  )

  # Aggregate duplicate GeneIDs.
  # Counts are summed.
  # Length must be unique/consistent for proper TPM calculation.
  duplicate_ids <- counts[
    duplicated(Geneid) | duplicated(Geneid, fromLast = TRUE),
    Geneid
  ]

  duplicate_lengths <- counts[
    Geneid %in% duplicate_ids,
    .(n_lengths = uniqueN(Length)),
    by = Geneid
  ]

  if (any(duplicate_lengths$n_lengths > 1)) {
    stop(
      "Some duplicated GeneIDs have different Length values. ",
      "These need to be resolved before TPM calculation."
    )
  }

  # Keep one annotation/length entry and sum counts
  counts <- counts[
    ,
    c(
      list(
        Chr = first(Chr),
        Start = first(Start),
        End = first(End),
        Strand = first(Strand),
        Length = first(Length)
```

```
      ),
      lapply(.SD, sum)
    ),
    by = Geneid,
    .SDcols = sample_cols
  ]
}

# ----------------------------
# Convert sample columns to numeric
# ----------------------------
counts[
  ,
  (sample_cols) := lapply(
    .SD,
    function(x) {
      x <- as.numeric(x)
      x[is.na(x)] <- 0
      x
    }
  ),
  .SDcols = sample_cols
]

# ------------------------------------------------------------------------------
# Remove genes with zero counts across all samples
# ------------------------------------------------------------------------------
keep <- rowSums(
  as.matrix(counts[, ..sample_cols])
) > 0

counts <- counts[keep]

cat(
  "\nGenes retained after removing all-zero genes:",
  nrow(counts),
  "\n"
)

# ------------------------------------------------------
# Save cleaned raw-count matrix
# ------------------------------------------------------

clean_counts <- counts[
  ,
  c("Geneid", "Length", sample_cols),
  with = FALSE
]

fwrite(
  clean_counts,
  clean_counts_file,
  sep = "\t",
  quote = FALSE,
  na = "NA"
```

```
)

cat(
  "\nCleaned raw counts saved to:\n",
  clean_counts_file,
  "\n"
)

# ----------------------------------------------------
# Calculate TPM
# ----------------------------------------------------

counts_tpm <- data.table(
  Geneid = counts$Geneid
)

for (s in sample_cols) {

  # Reads per kilobase
  rpk <- counts[[s]] / (counts$Length / 1000)

  # TPM scaling factor
  scaling_factor <- sum(rpk)

  if (scaling_factor == 0) {
    stop(
      "Sample ", s,
      " has zero total RPK."
    )
  }

  counts_tpm[[s]] <- rpk / scaling_factor * 1e6
}

# ----------------------------------------------------
# Check TPM sums
# ----------------------------------------------------

tpm_sums <- sapply(
  sample_cols,
  function(s) sum(counts_tpm[[s]])
)

cat("\nTPM sums:\n")
print(tpm_sums)

if (any(abs(tpm_sums - 1e6) > 1)) {
  warning(
    "One or more samples do not sum approximately to 1,000,000 TPM."
  )
}

# ----------------------------------------------------
# Save CIBERSORTx-ready TPM matrix
# ----------------------------------------------------
```

```
fwrite(
  counts_tpm,
  tpm_file,
  sep = "\t",
  quote = FALSE,
  na = "NA"
)

cat(
  "\nCIBERSORTx TPM matrix saved to:\n",
  tpm_file,
  "\n"
)

cat(
  "\nFinal matrix dimensions:",
  nrow(counts_tpm),
  "genes x",
  ncol(counts_tpm),
  "columns\n"
)

cat("\nFinal sample names:\n")
print(sample_cols)

cat("\nTPM preparation complete.\n")
```

Statistical analysis
CIBERSORTx immune response grouping

BASH script:

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --job-name=CIBERSORTx_immune_plots
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

set -e
set -o pipefail

echo "Job started on $(hostname)"

# Load miniconda (to get conda command)
export PATH="/opt/exp_soft/miniconda3-py3.12/bin:$PATH"
source /opt/exp_soft/miniconda3-py3.12/etc/profile.d/conda.sh

echo "Activating R conda environment..."
conda activate /scratch/[redacted]/conda_envs/conda_env

# Sanity check
echo "Which R am I using?"
which R
R --version

# Run the R script
echo "Running immune_group_final.R..."
Rscript immune_group_final.R

echo "Job finished"
```

R script:

```
# -------------------------
# Packages and directories
# -------------------------

library(tidyverse)
library(ggpubr)
library(rstatix)

# Output directory
outdir <- "/scratch/[redacted]/RNA_seq_analysis/CIBERSORTx"

dir.create(
  outdir,
  recursive = TRUE,
  showWarnings = FALSE
)

# -------------------------
# 1. Raw data
# -------------------------
immune <- data.frame(

  Sample = c(
    1406, 4270, 21498, 30752, 20329, 3916,
    25395, 14716, 29665, 22392, 18523, 10164, 13904
  ),

  Condition = c(
    "R", "R", "R", "R", "R", "R",
    "NR", "NR", "NR", "NR", "NR", "NR", "NR"
  ),

  Adaptive = c(
    7.006863119,
    4.897187512,
    5.124224327,
    6.542753898,
    5.796681424,
    5.315987910,
    3.998065857,
    2.743463867,
    5.100408385,
    2.680296816,
    7.316662023,
    6.532149154,
    3.299855702
  ),

  Total_T = c(
    2.069793193,
    2.970240972,
    1.731316453,
    2.335048494,
```

```
    1.160852691,
    1.464002218,
    0.871784839,
    1.278559609,
    3.060008646,
    1.077429818,
    6.546173740,
    2.743604080,
    2.691030673
  ),

  Cytotoxic = c(
    0.277750515,
    0.727832231,
    0.665143817,
    0.771473877,
    0.077294971,
    0.240596349,
    0.660160682,
    1.424232615,
    1.159757373,
    0.404984958,
    0.342814864,
    0.422189877,
    1.047129343
  ),

  Myeloid = c(
    1.099485995,
    1.825708901,
    1.567713898,
    0.665202009,
    1.262733533,
    0.919999894,
    0.557382257,
    2.076543373,
    2.376813907,
    0.631480094,
    3.367302616,
    0.556220203,
    2.138131603
  )
)

# Ensure factor order
immune$Condition <- factor(
  immune$Condition,
  levels = c("R", "NR")
)

# -------------------------
# 2. Long format
# -------------------------
immune_long <- immune %>%
  pivot_longer(
```

```
    cols = c(
      Adaptive,
      Total_T,
      Cytotoxic,
      Myeloid
    ),
    names_to = "Response",
    values_to = "Value"
  )

# Set desired boxplot order
immune_long$Response <- factor(
  immune_long$Response,
  levels = c(
    "Adaptive",
    "Total_T",
    "Cytotoxic",
    "Myeloid"
  )
)

# -------------------------
# 3. Descriptive statistics
# -------------------------
descriptive_stats <- immune_long %>%
  group_by(Response, Condition) %>%
  summarise(
    n = n(),
    Mean = mean(Value, na.rm = TRUE),
    SD = sd(Value, na.rm = TRUE),
    Median = median(Value, na.rm = TRUE),
    IQR = IQR(Value, na.rm = TRUE),
    Min = min(Value, na.rm = TRUE),
    Max = max(Value, na.rm = TRUE),
    .groups = "drop"
  )

# -------------------------
# 4. Wilcoxon test
# -------------------------
stat.test <- immune_long %>%
  group_by(Response) %>%
  wilcox_test(
    Value ~ Condition
  ) %>%
  adjust_pvalue(
    method = "BH"
  ) %>%
  add_significance()

# -------------------------
# 5. Combine statistics
# -------------------------
# Add R descriptive statistics
stats_R <- descriptive_stats %>%
```

```
  filter(Condition == "R") %>%
  select(
    Response,
    n_R = n,
    Mean_R = Mean,
    SD_R = SD,
    Median_R = Median,
    IQR_R = IQR,
    Min_R = Min,
    Max_R = Max
  )

# Add NR descriptive statistics
stats_NR <- descriptive_stats %>%
  filter(Condition == "NR") %>%
  select(
    Response,
    n_NR = n,
    Mean_NR = Mean,
    SD_NR = SD,
    Median_NR = Median,
    IQR_NR = IQR,
    Min_NR = Min,
    Max_NR = Max
  )

# Combine with Wilcoxon results
statistics_final <- stat.test %>%
  select(
    Response,
    statistic,
    p,
    p.adj,
    p.adj.signif
  ) %>%
  left_join(
    stats_R,
    by = "Response"
  ) %>%
  left_join(
    stats_NR,
    by = "Response"
  ) %>%
  select(
    Response,
    n_R,
    Mean_R,
    SD_R,
    Median_R,
    IQR_R,
    Min_R,
    Max_R,
    n_NR,
    Mean_NR,
    SD_NR,
```

```
    Median_NR,
    IQR_NR,
    Min_NR,
    Max_NR,
    statistic,
    p,
    p.adj,
    p.adj.signif
  )

# ------------------------
# 6. Save statistics
# ------------------------

stats_tsv <- file.path(
  outdir,
  "Immune_Composition_R_vs_NR_statistics.tsv"
)

stats_csv <- file.path(
  outdir,
  "Immune_Composition_R_vs_NR_statistics.csv"
)

write.table(
  statistics_final,
  file = stats_tsv,
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)

write.csv(
  statistics_final,
  file = stats_csv,
  quote = FALSE,
  row.names = FALSE
)

cat(
  "Statistics saved to:\n",
  stats_tsv,
  "\n",
  stats_csv,
  "\n"
)

# ------------------------
# 7. Position p-values
# ------------------------
stat.test <- stat.test %>%
  add_xy_position(
    x = "Response",
    step.increase = 0.05
  )
```

```
# -------------------------
# 8. Plot
# -------------------------
p <- ggplot(
  immune_long,
  aes(
    x = Response,
    y = Value,
    fill = Condition
  )
) +

  geom_boxplot(
    position = position_dodge(width = 0.75),
    width = 0.6,
    alpha = 0.8,
    outlier.shape = NA
  ) +

  # Individual sample points
  geom_point(
    aes(group = Condition),
    position = position_jitterdodge(
      jitter.width = 0.12,
      dodge.width = 0.75
    ),
    size = 2.5,
    shape = 16,
    colour = "black"
  ) +

  scale_fill_manual(
    values = c(
      "R" = "#E15759",
      "NR" = "#4E79A7"
    )
  ) +

  labs(
    title = "Immune Cell Composition: Rearranged vs Non-Rearranged",
    x = "",
    y = "Relative Immune Cell Abundance"
  ) +

  theme_classic(base_size = 14) +

  theme(
    axis.text.x = element_text(
      angle = 25,
      hjust = 1
    ),
    legend.title = element_blank(),
    plot.title = element_text(
      hjust = 0.5,
```

```
      face = "bold"
    )
  )

# -------------------------
# 9. Add p-values
# -------------------------
final_plot <- p +
  stat_pvalue_manual(
    stat.test,
    label = "p.adj.signif",
    tip.length = 0.01
  )

# -------------------------
# 10. Save plot
# -------------------------
outfile <- file.path(
  outdir,
  "Immune_Boxplot_R_vs_NR.png"
)

ggsave(
  filename = outfile,
  plot = final_plot,
  width = 9,
  height = 6,
  dpi = 600
)

cat(
  "Plot saved to:",
  outfile,
  "\n"
)
```

xCell cell type grouping

BASH script:

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --job-name=xCell_plots
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

set -e
set -o pipefail

echo "Job started on $(hostname)"

# Load miniconda (to get conda command)
export PATH="/opt/exp_soft/miniconda3-py3.12/bin:$PATH"
source /opt/exp_soft/miniconda3-py3.12/etc/profile.d/conda.sh

echo "Activating R conda environment..."
conda activate /scratch/[redacted]/conda_envs/conda_env

# Sanity check
echo "Which R am I using?"
which R
R --version

# Run the R script
echo "Running cell_typ_final.R..."
Rscript cell_type_final.R

echo "Job finished"
```

R script:

```
#!/usr/bin/env Rscript

# ------------------------------------------------------------
# 1. Load packages
# ------------------------------------------------------------
suppressPackageStartupMessages({

  library(tidyverse)
  library(rstatix)
  library(ggpubr)

})


# ------------------------------------------------------------
# 2. Output directory
# ------------------------------------------------------------
outdir <- "/scratch/[redacted]/RNA_seq_analysis/xCell"

dir.create(
  outdir,
  recursive = TRUE,
  showWarnings = FALSE
)


# ------------------------------------------------------------
# 3. Manually enter xCell scores
# ------------------------------------------------------------
xcell_stromal <- data.frame(

  Sample = c(
    "1406",
    "25395",
    "14716",
    "29665",
    "22392",
    "18523",
    "10164",
    "13904",
    "20329",
    "21498",
    "30752",
    "3916",
    "4270"
  ),

  Fibroblasts = c(
    0,
    0,
    0,
    0,
    0.0581,
```

```
  0,
  0,
  0.2066,
  0,
  0.0098,
  0,
  0,
  0
),

MSC = c(
  0.3035,
  0.0966,
  0.3322,
  0.3234,
  0.3218,
  0,
  0.2232,
  0.1989,
  0.2007,
  0.2389,
  0.3536,
  0.2420,
  0.1345
),

Pericytes = c(
  0,
  0.0571,
  0.0587,
  0.0475,
  0.0195,
  0,
  0.0384,
  0.0547,
  0,
  0,
  0,
  0,
  0
),

`Endothelial cells` = c(
  0.0566,
  0.0179,
  0.0217,
  0.0390,
  0.1360,
  0,
  0,
  0.1232,
  0.0462,
  0.1827,
  0.0828,
  0.0391,
```

```
    0.1307
  ),

  `mv Endothelial cells` = c(
    0.1070,
    0.1874,
    0.1652,
    0.1712,
    0.1763,
    0,
    0.1312,
    0.2213,
    0.0792,
    0.1959,
    0.1382,
    0.1215,
    0.1691
  ),

  `ly Endothelial cells` = c(
    0.0576,
    0.0623,
    0.0525,
    0.0467,
    0.1163,
    0,
    0.0522,
    0.1389,
    0.0447,
    0.1030,
    0.0550,
    0.0621,
    0.0950
  ),

  stringsAsFactors = FALSE,

  check.names = FALSE

)


# ----------------------------------------------------------
# 4. Define selected cell populations
# ----------------------------------------------------------
selected_cells <- c(
  "Fibroblasts",
  "MSC",
  "Pericytes",
  "Endothelial cells",
  "mv Endothelial cells",
  "ly Endothelial cells"
)
```

```
# ----------------------------------------------------------
# 5. Verify that all six populations exist
# ----------------------------------------------------------
missing_cells <- setdiff(
  selected_cells,
  colnames(xcell_stromal)
)


if (length(missing_cells) > 0) {

  stop(
    paste0(
      "\nERROR: The following selected populations ",
      "are missing from the manually entered data:\n\n",
      paste(
        missing_cells,
        collapse = "\n"
      )
    )
  )

}


# ----------------------------------------------------------
# 6. Define R and NR samples
# ----------------------------------------------------------
R_samples <- c(
  "20329",
  "21498",
  "30752",
  "3916",
  "4270",
  "1406"
)


NR_samples <- c(
  "25395",
  "14716",
  "29665",
  "22392",
  "18523",
  "10164",
  "13904"
)


# ----------------------------------------------------------
# 7. Assign conditions
# ----------------------------------------------------------
xcell_stromal <- xcell_stromal %>%

  mutate(
```

```
    Condition = case_when(

      Sample %in% R_samples ~ "R",

      Sample %in% NR_samples ~ "NR",

      TRUE ~ NA_character_

    )

  )


# ------------------------------------------------------------
# 8. Check for unassigned samples
# ------------------------------------------------------------

unassigned <- xcell_stromal %>%

  filter(
    is.na(Condition)
  ) %>%

  pull(
    Sample
  )


if (length(unassigned) > 0) {

  stop(
    paste0(
      "\nERROR: The following samples could not be ",
      "assigned to R or NR:\n\n",
      paste(
        unassigned,
        collapse = ", "
      )
    )
  )

}


# ------------------------------------------------------------
# 9. Set condition order
# ------------------------------------------------------------
xcell_stromal$Condition <- factor(

  xcell_stromal$Condition,

  levels = c(
    "R",
    "NR"
```

```
  )

)


# ------------------------------------------------------
# 10. Print sample grouping
# ------------------------------------------------------
cat("\n")
cat("============================================================\n")
cat("SAMPLE GROUPING\n")
cat("============================================================\n\n")


cat(
  "R samples (",
  sum(
    xcell_stromal$Condition == "R"
  ),
  "): ",
  sep = ""
)


cat(
  paste(
    xcell_stromal$Sample[
      xcell_stromal$Condition == "R"
    ],
    collapse = ", "
  ),
  "\n"
)


cat(
  "NR samples (",
  sum(
    xcell_stromal$Condition == "NR"
  ),
  "): ",
  sep = ""
)


cat(
  paste(
    xcell_stromal$Sample[
      xcell_stromal$Condition == "NR"
    ],
    collapse = ", "
  ),
  "\n\n"
)
```

```
# ------------------------------------------------------------
# 11. Verify sample counts
# ------------------------------------------------------------
if (
  sum(
    xcell_stromal$Condition == "R"
  ) != 6
) {

  stop(
    paste0(
      "ERROR: Expected 6 R samples but found ",
      sum(
        xcell_stromal$Condition == "R"
      ),
      "."
    )
  )

}


if (
  sum(
    xcell_stromal$Condition == "NR"
  ) != 7
) {

  stop(
    paste0(
      "ERROR: Expected 7 NR samples but found ",
      sum(
        xcell_stromal$Condition == "NR"
      ),
      "."
    )
  )

}


# ------------------------------------------------------------
# 12. Long format
# ------------------------------------------------------------
xcell_long <- xcell_stromal %>%

  pivot_longer(

    cols = all_of(
      selected_cells
    ),

    names_to = "Cell_Type",
```

```
    values_to = "Score"

  )


xcell_long$Cell_Type <- factor(

  xcell_long$Cell_Type,

  levels = selected_cells

)


# ------------------------------------------------------------
# 13. Check missing values
# ------------------------------------------------------------
missing_summary <- xcell_long %>%

  group_by(
    Cell_Type
  ) %>%

  summarise(

    Missing = sum(
      is.na(Score)
    ),

    .groups = "drop"

  )


cat("\n")
cat("Missing values:\n")
print(
  missing_summary
)
cat("\n")


# ------------------------------------------------------------
# 14. Wilcoxon rank-sum testing
# ------------------------------------------------------------
stat.test <- xcell_long %>%

  group_by(
    Cell_Type
  ) %>%

  wilcox_test(

    Score ~ Condition,
```

```
    exact = FALSE

  )


# ----------------------------------------------------------
# 15. Benjamini-Hochberg corrections
# ----------------------------------------------------------

stat.test <- stat.test %>%

  adjust_pvalue(

    method = "BH"

  ) %>%

  add_significance(
    "p.adj"
  )


# ----------------------------------------------------------
# 16. Summary statistics
# ----------------------------------------------------------
summary_data <- xcell_long %>%

  group_by(
    Cell_Type,
    Condition
  ) %>%

  summarise(

    Mean = mean(
      Score,
      na.rm = TRUE
    ),

    SD = sd(
      Score,
      na.rm = TRUE
    ),

    Median = median(
      Score,
      na.rm = TRUE
    ),

    N = sum(
      !is.na(Score)
    ),

    .groups = "drop"
```

```
  )


# ----------------------------------------------------------
# 17. Summary
# ----------------------------------------------------------
summary_wide <- summary_data %>%

  pivot_wider(

    names_from = Condition,

    values_from = c(
      Mean,
      SD,
      Median,
      N
    )

  )


# ----------------------------------------------------------
# 18. Combine statistical results with summary data
# ----------------------------------------------------------

results_table <- stat.test %>%

  select(
    Cell_Type,
    p,
    p.adj,
    p.adj.signif
  ) %>%

  left_join(
    summary_wide,
    by = "Cell_Type"
  )


results_table$Cell_Type <- factor(

  results_table$Cell_Type,

  levels = selected_cells

)


results_table <- results_table %>%

  arrange(
    Cell_Type
  )
```

```
# ------------------------------------------------------------
# 19. Save statistical results
# ------------------------------------------------------------
stats_file <- file.path(

  outdir,

  "xCell_six_stromal_R_vs_NR_statistics.csv"

)


write.csv(

  results_table,

  stats_file,

  row.names = FALSE

)


# ------------------------------------------------------------
# 20. Print statistical results
# ------------------------------------------------------------
cat("\n")
cat("============================================================\n")
cat("STATISTICAL RESULTS\n")
cat("============================================================\n\n")


print(
  results_table
)


# ------------------------------------------------------------
# 21. Calculate significance-bar positions
# ------------------------------------------------------------
y_positions <- xcell_long %>%

  group_by(
    Cell_Type
  ) %>%

  summarise(

    ymax = max(
      Score,
      na.rm = TRUE
    ),
```

```
    y.position = ifelse(
      ymax > 0,
      ymax * 1.20,
      0.05
    ),
    .groups = "drop"
  )

# ------------------------------------------------------------
# 22. Prepare significance-bar positions
# ------------------------------------------------------------
stat.test <- stat.test %>%
  left_join(
    y_positions,
    by = "Cell_Type"
  ) %>%
  mutate(
    Cell_Type = factor(
      Cell_Type,
      levels = selected_cells
    ),
    x.position = as.numeric(
      Cell_Type
    ),
    xmin = x.position - 0.20,
    xmax = x.position + 0.20
  )

# ------------------------------------------------------------
# 23. Create boxplot
# ------------------------------------------------------------
p <- ggplot(
```

```
  xcell_long,
  aes(
    x = Cell_Type,
    y = Score,
    fill = Condition
  )
) +
  geom_boxplot(
    position = position_dodge(
      width = 0.75
    ),
    width = 0.60,
    alpha = 0.8,
    outlier.shape = NA
  ) +
  geom_jitter(
    position = position_jitterdodge(
      jitter.width = 0.08,
      dodge.width = 0.75
    ),
    size = 2.5,
    alpha = 0.9,
    colour = "black",
    show.legend = FALSE
  ) +
  stat_boxplot(
    geom = "errorbar",
    position = position_dodge(
```

```
    width = 0.75
  ),
  width = 0.20
) +
scale_fill_manual(
  values = c(
    "R" = "#E15759",
    "NR" = "#4E79A7"
  )
) +
stat_pvalue_manual(
  stat.test,
  label = "p.adj.signif",
  xmin = "xmin",
  xmax = "xmax",
  y.position = "y.position",
  tip.length = 0.01,
  hide.ns = FALSE
) +
labs(
  title =
    "xCell Stromal/Vascular Enrichment: R vs NR",
  x = "",
  y =
    "xCell enrichment score"
) +
theme_classic(
  base_size = 14
```

```
  ) +
  theme(
    axis.text.x =
      element_text(
        angle = 45,
        hjust = 1
      ),
    legend.title =
      element_blank(),
    plot.title =
      element_text(
        hjust = 0.5,
        face = "bold"
      )
  )

# ------------------------------------------------------------
# 24. Save boxplot
# ------------------------------------------------------------
plot_file <- file.path(
  outdir,
  "xCell_six_stromal_R_vs_NR_boxplot.png"
)

ggsave(
  filename = plot_file,
  plot = p,
  width = 11,
  height = 6.5,
  dpi = 600
)
```

```
# ------------------------------------------------------------
# 25. Save six population data
# ------------------------------------------------------------
selected_data_file <- file.path(

  outdir,

  "xCell_six_stromal_populations_used.csv"

)


write.csv(

  xcell_stromal,

  selected_data_file,

  row.names = FALSE

)


# ------------------------------------------------------------
# 26. Save long format data
# ------------------------------------------------------------
long_data_file <- file.path(

  outdir,

  "xCell_six_stromal_populations_long_format.csv"

)


write.csv(

  xcell_long,

  long_data_file,

  row.names = FALSE

)


# ------------------------------------------------------------
# 27. Final message
# ------------------------------------------------------------
cat("\n")
cat("============================================================\n")
cat("ANALYSIS COMPLETE\n")
cat("============================================================\n\n")
```

```
cat(
  "Six populations tested:\n\n"
)


cat(
  paste(
    selected_cells,
    collapse = "\n"
  ),
  "\n\n"
)


cat(
  "Statistical test:\n",
  "Wilcoxon rank-sum test\n\n"
)


cat(
  "Multiple-testing correction:\n",
  "Benjamini-Hochberg across SIX tests only\n\n"
)


cat(
  "Statistics saved to:\n",
  stats_file,
  "\n\n"
)


cat(
  "Boxplot saved to:\n",
  plot_file,
  "\n\n"
)


cat(
  "Six-population data saved to:\n",
  selected_data_file,
  "\n\n"
)


cat(
  "Long-format data saved to:\n",
  long_data_file,
  "\n\n"
)
```

Correlation analysis between tissue age, RIN and DV200

BASH script:

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --job-name=correlation_analysis
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

set -e
set -o pipefail

# Activate environment (assuming it exists)
export PATH="/opt/exp_soft/miniconda3-py3.12/bin:$PATH"
source /opt/exp_soft/miniconda3-py3.12/etc/profile.d/conda.sh
echo "Activating environment..."
conda activate /scratch/[redacted]/conda_envs/conda_env

# Run Python script
echo "Running correlations_final.py..."
python correlations_final.py
```

Python script:

```
# ----------------------------
# Importing packages
# ----------------------------
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import pearsonr, spearmanr

# ----------------------------
# Output directory
# ----------------------------
out_dir = "/scratch/[redacted]/RNA_seq_analysis/rna_qc/correlations_RIN_DV200"
os.makedirs(out_dir, exist_ok=True)

# ----------------------------
# Data
# ----------------------------
data = {
   "Age":   [31, 44, 81, 56, 52, 45, 76, 69, 37, 57, 65, 54, 79],
   "RIN":   [1.8, 2, 2.5, 2.6, 2.1, 1.6, 2.2, 2.3, 2.7, 2.8, 2.1, 2.5, 2.4],
   "DV200": [55.3, 44.34, 33.32, 39.34, 60.42, 41.13, 49.11, 58.39, 67.72, 57.45, 74.04,
64.48, 39.98]
}

df = pd.DataFrame(data)

# ----------------------------
# Correlations
# ----------------------------
r_rin, p_rin = pearsonr(df["Age"], df["RIN"])
rho_dv200, p_dv200 = spearmanr(df["Age"], df["DV200"])

# ---------------------------------------
# PLOT 1: Age vs RIN (Pearson analysis)
# ---------------------------------------
plt.figure()
plt.scatter(df["Age"], df["RIN"])
plt.xlabel("Age")
plt.ylabel("RIN")
plt.title(f"Age vs RIN (Pearson r={r_rin:.2f}, p={p_rin:.2e})")

# simple regression line
m, b = np.polyfit(df["Age"], df["RIN"], 1)
plt.plot(df["Age"], m*df["Age"] + b)

plt.tight_layout()
plt.savefig(os.path.join(out_dir, "Age_vs_RIN_scatter.png"), dpi=300)
plt.close()

# ---------------------------------------
# PLOT 2: Age vs DV200 (Spearman analysis)
# ---------------------------------------
```

```
plt.figure()
plt.scatter(df["Age"], df["DV200"])
plt.xlabel("Age")
plt.ylabel("DV200")
plt.title(f"Age vs DV200 (Spearman ρ={rho_dv200:.2f}, p={p_dv200:.2e})")

plt.tight_layout()
plt.savefig(os.path.join(out_dir, "Age_vs_DV200_scatter.png"), dpi=300)
plt.close()

# ----------------------------------------
# Save results table
# ----------------------------------------
results = {
    "Age_vs_RIN_r": r_rin,
    "Age_vs_RIN_p": p_rin,
    "Age_vs_DV200_rho": rho_dv200,
    "Age_vs_DV200_p": p_dv200
}

pd.DataFrame([results]).to_csv(
    os.path.join(out_dir, "correlation_results.csv"),
    index=False
)

print("Done. Outputs saved to:", out_dir)
```

Difference of *BCL6* gene expression between BCL6-positive and BCL6-negative groups

BASH script:

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --job-name=IHC_BCL6_expression
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

set -e
set -o pipefail

echo "Job started on $(hostname)"

# Load miniconda (to get conda command)
export PATH="/opt/exp_soft/miniconda3-py3.12/bin:$PATH"
source /opt/exp_soft/miniconda3-py3.12/etc/profile.d/conda.sh

echo "Activating R conda environment..."
conda activate /scratch/[redacted]/conda_envs/conda_env

# Sanity check
echo "Which R am I using?"
which R
R --version

# Run the R script
echo "Running BCL6_IHC_final.R..."
Rscript BCL6_IHC_final.R

echo "Job finished"
```

R script:

```
#!/usr/bin/env Rscript

suppressPackageStartupMessages({
  library(tidyverse)
  library(ggpubr)
  library(rstatix)
})

# ------------------------------------------------------------
# 1. Load VST matrix
# ------------------------------------------------------------

vst_file <- "/scratch/[redacted]/RNA_seq_analysis/DESeq2/vst_counts.tsv"

vst <- read.delim(
  vst_file,
  header = TRUE,
  sep = "\t",
  row.names = 1,
  check.names = FALSE,
  stringsAsFactors = FALSE
)

# ------------------------------------------------------------
# 2. Define IHC-BCL6 groups
# ------------------------------------------------------------

ihc_positive <- c(
  "4270",
  "21498",
  "20329",
  "3916",
  "14716",
  "18523",
  "13904"
)

ihc_negative <- c(
  "1406",
  "30752",
  "25395",
  "29665",
  "22392",
  "10164"
)

# ------------------------------------------------------------
# 3. Gene of interest
# ------------------------------------------------------------

gene <- "BCL6"

if (!(gene %in% rownames(vst))) {
```

```
  stop(paste("Gene", gene, "not found in VST matrix."))
}

# ------------------------------------------------------------
# 4. Check that all required samples are present
# ------------------------------------------------------------

required_samples <- c(
  ihc_positive,
  ihc_negative
)

missing_samples <- setdiff(
  required_samples,
  colnames(vst)
)

if (length(missing_samples) > 0) {
  stop(
    paste(
      "The following required samples are missing from the VST matrix:",
      paste(missing_samples, collapse = ", ")
    )
  )
}

# ------------------------------------------------------------
# 5. Extract expression data
# ------------------------------------------------------------

expression_data <- vst[
  gene,
  required_samples,
  drop = FALSE
]

expression_long <- expression_data %>%
  as.data.frame() %>%
  rownames_to_column("gene") %>%
  pivot_longer(
    cols = -gene,
    names_to = "Sample",
    values_to = "Expression"
  ) %>%
  mutate(
    IHC_BCL6 = case_when(
      Sample %in% ihc_positive ~ "Positive",
      Sample %in% ihc_negative ~ "Negative",
      TRUE ~ NA_character_
    ),
    IHC_BCL6 = factor(
      IHC_BCL6,
      levels = c("Positive", "Negative")
    )
  ) %>%
```

```
  filter(!is.na(IHC_BCL6))

# ------------------------------------------------------------
# 6. Statistical analysis
# ------------------------------------------------------------

wilcox_test <- expression_long %>%
  wilcox_test(
    Expression ~ IHC_BCL6,
    exact = FALSE
  ) %>%
  add_significance(
    p.col = "p",
    cutpoints = c(
      0,
      0.0001,
      0.001,
      0.01,
      0.05,
      Inf
    ),
    symbols = c(
      "****",
      "***",
      "**",
      "*",
      "ns"
    )
  ) %>%
  add_xy_position(
    x = "IHC_BCL6",
    fun = "max"
  )

# ------------------------------------------------------------
# 7. Summary statistics
# ------------------------------------------------------------

summary_stats <- expression_long %>%
  group_by(IHC_BCL6) %>%
  summarise(
    N = n(),
    Mean = mean(Expression, na.rm = TRUE),
    SD = sd(Expression, na.rm = TRUE),
    Median = median(Expression, na.rm = TRUE),
    Q1 = quantile(Expression, 0.25, na.rm = TRUE),
    Q3 = quantile(Expression, 0.75, na.rm = TRUE),
    IQR = IQR(Expression, na.rm = TRUE),
    .groups = "drop"
  )

# Add Wilcoxon statistics and significance
summary_stats <- summary_stats %>%
  mutate(
    W = wilcox_test$statistic,
```

```
    p = wilcox_test$p,
    Significance = wilcox_test$p.signif
  )

# ------------------------------------------------------------
# 8. Create plot
# ------------------------------------------------------------

p <- ggplot(
  expression_long,
  aes(
    x = IHC_BCL6,
    y = Expression,
    fill = IHC_BCL6
  )
) +
  geom_boxplot(
    width = 0.6,
    alpha = 0.8,
    outlier.shape = NA
  ) +
  geom_jitter(
    width = 0.08,
    size = 2.5,
    shape = 16,
    colour = "black",
    show.legend = FALSE
  ) +
  scale_fill_manual(
    values = c(
      "Positive" = "#E15759",
      "Negative" = "#4E79A7"
    )
  ) +
  labs(
    title = paste(gene, "Expression by IHC-BCL6 Status"),
    x = "IHC-BCL6 Status",
    y = paste(gene, "VST Expression")
  ) +
  theme_classic(base_size = 14) +
  theme(
    legend.position = "none",
    plot.title = element_text(hjust = 0.5, face = "bold")
  )

# ------------------------------------------------------------
# 9. Add statistical annotation
# ------------------------------------------------------------

final_plot <- p +
  stat_pvalue_manual(
    wilcox_test,
    label = "p.signif",
    tip.length = 0.01
  )
```

```
# ------------------------------------------------------------
# 10. Output directory
# ------------------------------------------------------------

output_dir <- file.path(
  "/scratch/[redacted]/RNA_seq_analysis/DESeq2/expression_level_plots",
  gene
)

dir.create(
  output_dir,
  recursive = TRUE,
  showWarnings = FALSE
)

# ------------------------------------------------------------
# 11. Save plot
# ------------------------------------------------------------

ggsave(
  filename = file.path(
    output_dir,
    paste0(gene, "_IHC_BCL6_Positive_vs_Negative.png")
  ),
  plot = final_plot,
  width = 6,
  height = 6,
  dpi = 600
)

# ------------------------------------------------------------
# 12. Save statistics
# ------------------------------------------------------------

write.table(
  summary_stats,
  file = file.path(
    output_dir,
    paste0(gene, "_IHC_BCL6_summary_statistics.tsv")
  ),
  sep = "\t",
  row.names = FALSE,
  quote = FALSE
)

write.csv(
  summary_stats,
  file = file.path(
    output_dir,
    paste0(gene, "_IHC_BCL6_summary_statistics.csv")
  ),
  row.names = FALSE
)
```

```
# ------------------------------------------------------------
# 13. Print results
# ------------------------------------------------------------

print(final_plot)

cat("\nSummary statistics:\n")
print(summary_stats)
```

Difference of Ki67 index and *MKI* gene expression between *BCL6*-rearranged and non-rearranged groups

BASH script:

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --job-name=Ki67_MKI67_expression
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

set -e
set -o pipefail

echo "Job started on $(hostname)"

# Load miniconda (to get conda command)
export PATH="/opt/exp_soft/miniconda3-py3.12/bin:$PATH"
source /opt/exp_soft/miniconda3-py3.12/etc/profile.d/conda.sh

echo "Activating R conda environment..."
conda activate /scratch/[redacted]/conda_envs/conda_env

# Sanity check
echo "Which R am I using?"
which R
R --version

# Run the R script
echo "Running Ki67_MKI67_expression_final.R..."
Rscript Ki67_MKI67_expression_final.R

echo "Job finished"
```

R script:

```
# Load packages

library(tidyverse)
library(ggpubr)
library(rstatix)

# Output directory
outdir <- "/scratch/[redacted]/RNA_seq_analysis/Ki67"

# Create output directory if it does not exist
dir.create(outdir, recursive = TRUE, showWarnings = FALSE)


# -------------------------
# 1. Raw data
# -------------------------

ki67 <- data.frame(

  Sample = c(25395,14716,29665,22392,18523,10164,13904,
             20329,21498,30752,3916,4270,1406),

  Condition = c("NR","NR","NR","NR","NR","NR","NR",
                "R","R","R","R","R","R"),

  Ki67 = c(
    # NR samples (7)
    85,90,50,90,90,75,70,

    # R samples (6)
    85,60,70,75,70,65
  )
)


# Ensure factor order
ki67$Condition <- factor(
  ki67$Condition,
  levels = c("R","NR")
)


# -------------------------
# 2. Summary statistics
# -------------------------

summary_stats <- ki67 %>%
  group_by(Condition) %>%
  summarise(
    n = n(),
    Mean = mean(Ki67, na.rm = TRUE),
    SD = sd(Ki67, na.rm = TRUE),
    Median = median(Ki67, na.rm = TRUE),
```

```
    .groups = "drop"
  )

print(summary_stats)


# Save summary statistics
write.csv(
  summary_stats,
  file.path(outdir, "Ki67_R_vs_NR_summary_statistics.csv"),
  row.names = FALSE
)


# -------------------------
# 3. Statistical test
# -------------------------

stat.test <- ki67 %>%
  wilcox_test(Ki67 ~ Condition) %>%
  mutate(
    p.label = ifelse(
      p < 0.05,
      "p < 0.05",
      paste0("p = ", format.pval(p, digits = 3, eps = 0.001))
    )
  )

print(stat.test)


# Print exact p-value separately
cat(
  "\nWilcoxon rank-sum test p-value:",
  format.pval(stat.test$p, digits = 6, eps = 0.000001),
  "\n"
)


# Save statistical test results
write.csv(
  stat.test,
  file.path(outdir, "Ki67_R_vs_NR_Wilcoxon_test.csv"),
  row.names = FALSE
)


# Add y-position for p-value label
stat.test <- stat.test %>%
  add_xy_position(x = "Condition")


# -------------------------
# 4. Plot
# -------------------------
```

```
p <- ggplot(
  ki67,
  aes(
    x = Condition,
    y = Ki67,
    fill = Condition
  )
) +

  geom_boxplot(
    width = 0.5,
    alpha = 0.8,
    outlier.shape = NA
  ) +

  geom_jitter(
    width = 0.08,
    size = 2.5,
    alpha = 0.9,
    colour = "black",
    show.legend = FALSE
  ) +

  scale_fill_manual(
    values = c(
      "R" = "#E15759",
      "NR" = "#4E79A7"
    )
  ) +

  labs(
    title = "Ki-67 Expression (R vs NR)",
    x = "",
    y = "Ki-67 positive cells (%)"
  ) +

  theme_classic(base_size = 14) +

  theme(
    legend.title = element_blank(),
    plot.title = element_text(
      hjust = 0.5,
      face = "bold"
    )
  )


# Add significance label
final_plot <- p +
  stat_pvalue_manual(
    stat.test,
    label = "p.label",
    tip.length = 0.01
  )
```

```
# -------------------------
# 5. Save output
# -------------------------

outfile <- file.path(
  outdir,
  "Ki67_R_vs_NR_Boxplot.png"
)

ggsave(
  filename = outfile,
  plot = final_plot,
  width = 6,
  height = 6,
  dpi = 600
)


cat(
  "\nPlot saved to:",
  outfile,
  "\n"
)
```

Difference of *MYC* and *BCL6* gene expression between *BCL6*-rearranged and non-rearranged groups

BASH script:

```
#!/bin/bash
#SBATCH --account=pathology
#SBATCH --partition=ada
#SBATCH --time=10:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --job-name=MYC_BCL6_expression
#SBATCH --mail-user=[redacted]
#SBATCH --mail-type=ALL

set -e
set -o pipefail

echo "Job started on $(hostname)"

# Load miniconda (to get conda command)
export PATH="/opt/exp_soft/miniconda3-py3.12/bin:$PATH"
source /opt/exp_soft/miniconda3-py3.12/etc/profile.d/conda.sh

echo "Activating R conda environment..."
conda activate /scratch/[redacted]/conda_envs/conda_env

# Sanity check
echo "Which R am I using?"
which R
R --version

# Run the R script
echo "Running MYC_BCL6_expression_final.R..."
Rscript MYC_BCL6_expression_final.R

echo "Job finished"
```

R script:

```
# ------------------------------------------------
# 1. Load required packages
# ------------------------------------------------

library(tidyverse)
library(ggpubr)
library(rstatix)


# ------------------------------------------------
# 2. Input file
# ------------------------------------------------

vst_file <- "/scratch/[redacted]/RNA_seq_analysis/DESeq2/vst_counts.tsv"

vst <- read.delim(
  vst_file,
  row.names = 1,
  check.names = FALSE
)


# ------------------------------------------------
# 3. Define sample groups
# ------------------------------------------------

R_samples <- c(
  "1406",
  "21498",
  "30752",
  "20329",
  "4270",
  "3916"
)

NR_samples <- c(
  "18523",
  "13904",
  "29665",
  "10164",
  "14716",
  "25395",
  "22392"
)


# ------------------------------------------------
# 4. Define output directory
# ------------------------------------------------

outdir <- "/scratch/[redacted]/RNA_seq_analysis/DESeq2/expression_level_plots"

dir.create(
```

```
  outdir,
  showWarnings = FALSE,
  recursive = TRUE
)


# ----------------------------------------------------
# 5. Check that all required samples are present
# ----------------------------------------------------

required_samples <- c(
  R_samples,
  NR_samples
)

missing_samples <- setdiff(
  required_samples,
  colnames(vst)
)

if (length(missing_samples) > 0) {

  stop(
    paste(
      "The following required samples were not found in the VST matrix:",
      paste(missing_samples, collapse = ", ")
    )
  )

}


# Check that the required genes are present

required_genes <- c(
  "MYC",
  "BCL6"
)

missing_genes <- setdiff(
  required_genes,
  rownames(vst)
)

if (length(missing_genes) > 0) {

  stop(
    paste(
      "The following genes were not found in the VST matrix:",
      paste(missing_genes, collapse = ", ")
    )
  )

}
```

```
# ------------------------------------------------
# 6. Function for gene analysis
# ------------------------------------------------

analyse_gene <- function(
  gene,
  outdir,
  plot_filename,
  plot_title,
  y_axis_label
) {

  # ------------------------------------------------------
  # Extract gene expression
  # ------------------------------------------------------

  expression <- as.numeric(
    vst[
      gene,
      c(R_samples, NR_samples)
    ]
  )

  names(expression) <- c(
    R_samples,
    NR_samples
  )


  # ------------------------------------------------------
  # Convert to long format
  # ------------------------------------------------------

  expression_long <- data.frame(
    Sample = names(expression),
    Expression = expression
  ) %>%

    mutate(
      Condition = case_when(
        Sample %in% R_samples ~ "R",
        Sample %in% NR_samples ~ "NR",
        TRUE ~ NA_character_
      )
    ) %>%

    filter(
      !is.na(Condition)
    )


  # ------------------------------------------------------
  # Set factor order
  # ------------------------------------------------------
```

```r
expression_long$Condition <- factor(
  expression_long$Condition,
  levels = c("R", "NR")
)


# ------------------------------------------------
# 7. Summary statistics
# ------------------------------------------------

summary_stats <- expression_long %>%

  group_by(Condition) %>%

  summarise(
    n = n(),
    Mean = mean(
      Expression,
      na.rm = TRUE
    ),
    SD = sd(
      Expression,
      na.rm = TRUE
    ),
    Median = median(
      Expression,
      na.rm = TRUE
    ),
    .groups = "drop"
  )


cat(
  "\n========================================\n",
  "Summary statistics for ", gene, "\n",
  "========================================\n",
  sep = ""
)

print(summary_stats)


# ------------------------------------------------
# 8. Wilcoxon rank-sum test
# ------------------------------------------------

stat.test <- expression_long %>%

  wilcox_test(
    Expression ~ Condition,
    exact = FALSE
  )
```

```
# Extract unadjusted p-value

p_value <- stat.test$p


# Significance label based on unadjusted p-value

significance_label <- ifelse(
  p_value < 0.05,
  "*",
  "ns"
)


# Add significance information to test results

stat.test <- stat.test %>%

  mutate(
    Gene = gene,
    significance = significance_label,
    p.label = paste0(
      "p = ",
      format.pval(
        p_value,
        digits = 3,
        eps = 0.001
      )
    )
  ) %>%

  add_xy_position(
    x = "Condition"
  )


cat(
  "\n========================================\n",
  "Wilcoxon rank-sum test for ", gene, "\n",
  "========================================\n",
  sep = ""
)

print(stat.test)


# ------------------------------------------------------------
# 9. Combine summary statistics and p-value
# ------------------------------------------------------------

statistical_summary <- summary_stats %>%

  select(
    Condition,
    n,
```

```
    Mean,
    SD,
    Median
  ) %>%

  pivot_wider(
    names_from = Condition,
    values_from = c(
      n,
      Mean,
      SD,
      Median
    ),
    names_glue = "{.value}_{Condition}"
  ) %>%

  mutate(
    Gene = gene,
    Wilcoxon_p = p_value,
    Significance_unadjusted = significance_label
  ) %>%

  select(
    Gene,
    n_R,
    Mean_R,
    SD_R,
    Median_R,
    n_NR,
    Mean_NR,
    SD_NR,
    Median_NR,
    Wilcoxon_p,
    Significance_unadjusted
  )


cat(
  "\n========================================\n",
  "Statistical summary for ", gene, "\n",
  "========================================\n",
  sep = ""
)

print(statistical_summary)


# ------------------------------------------------------
# 10. Save individual statistical summary
# ------------------------------------------------------

write.csv(
  statistical_summary,
  file.path(
    outdir,
```

```
    paste0(
      gene,
      "_R_vs_NR_statistics.csv"
    )
  ),
  row.names = FALSE
)


# ------------------------------------------------------------
# 11. Create boxplot
# ------------------------------------------------------------

p <- ggplot(
  expression_long,
  aes(
    x = Condition,
    y = Expression,
    fill = Condition
  )
) +

  geom_boxplot(
    width = 0.6,
    alpha = 0.8,
    outlier.shape = NA
  ) +

  geom_jitter(
    width = 0.08,
    size = 2.5,
    alpha = 0.9
  ) +

  scale_fill_manual(
    values = c(
      "R" = "#E15759",
      "NR" = "#4E79A7"
    )
  ) +

  labs(
    title = plot_title,
    x = "",
    y = y_axis_label
  ) +

  theme_classic(
    base_size = 14
  ) +

  theme(
    legend.position = "none",
    plot.title = element_text(
      hjust = 0.5,
```

```
      face = "bold"
    )
  )


# ------------------------------------------------------------
# 12. Add statistical bracket
# ------------------------------------------------------------

final_plot <- p +

  stat_pvalue_manual(
    stat.test,
    label = "p.label",
    tip.length = 0.01,
    hide.ns = FALSE
  )


# ------------------------------------------------------------
# 13. Save individual boxplot
# ------------------------------------------------------------

ggsave(
  filename = file.path(
    outdir,
    plot_filename
  ),
  plot = final_plot,
  width = 6,
  height = 6,
  dpi = 600
)


cat(
  "\nPlot saved to:\n",
  file.path(
    outdir,
    plot_filename
  ),
  "\n",
  sep = ""
)


# ------------------------------------------------------------
# 14. Return statistical results
# ------------------------------------------------------------

return(statistical_summary)

}
```

```
# ------------------------------------------------
# 15. MYC analysis
# ------------------------------------------------

MYC_results <- analyse_gene(
  gene = "MYC",
  outdir = outdir,
  plot_filename = "MYC_R_vs_NR_VST_boxplot.png",
  plot_title = "MYC Expression (R vs NR)",
  y_axis_label = "MYC VST Expression"
)


# ------------------------------------------------
# 16. BCL6 analysis
# ------------------------------------------------

BCL6_results <- analyse_gene(
  gene = "BCL6",
  outdir = outdir,
  plot_filename = "BCL6_R_vs_NR_VST_boxplot.png",
  plot_title = "BCL6 Expression (R vs NR)",
  y_axis_label = "BCL6 VST Expression"
)


# ------------------------------------------------
# 17. Combine results
# ------------------------------------------------

combined_results <- bind_rows(
  MYC_results,
  BCL6_results
)


# ------------------------------------------------
# 18. Apply Benjamini-Hochberg correction
# ------------------------------------------------

combined_results <- combined_results %>%

  mutate(
    Wilcoxon_padj = p.adjust(
      Wilcoxon_p,
      method = "BH"
    ),

    Significance_adjusted = ifelse(
      Wilcoxon_padj < 0.05,
      "*",
      "ns"
    )
  )
```

```
# ------------------------------------------------
# 19. Display combined results
# ------------------------------------------------

cat(
  "\n========================================\n",
  "COMBINED STATISTICAL RESULTS\n",
  "========================================\n",
  sep = ""
)

print(combined_results)


# ------------------------------------------------
# 20. Save combined results
# ------------------------------------------------

combined_file <- file.path(
  outdir,
  "MYC_BCL6_R_vs_NR_statistics.csv"
)

write.csv(
  combined_results,
  combined_file,
  row.names = FALSE
)


cat(
  "\nCombined results saved to:\n",
  combined_file,
  "\n",
  sep = ""
)


# ------------------------------------------------
# 21. Final output message
# ------------------------------------------------

cat(
  "\n========================================\n"
)

cat(
  "ANALYSIS COMPLETE\n"
)

cat(
  "Two individual gene expression plots generated:\n"
)
```

```
cat(
  "  - MYC\n"
)

cat(
  "  - BCL6\n"
)

cat(
  "\nIndividual and combined statistical results saved.\n"
)

cat(
  "\nAll output files are located in:\n"
)

cat(
  outdir,
  "\n"
)

cat(
  "========================================\n"
```